

## ✓ Multi-classification - Water Quality Classification

**Project Type** - Multi-classification

**Contribution** - Individual

**Name** - Manoj B

## ✓ Project Summary -

### 1. Project Objective

This project aims to develop a machine learning model to classify water bodies in Maharashtra, India, into specific quality categories (Class A, B, C, E). The primary goal is to automate the assessment of water safety by analyzing physicochemical and biological parameters—such as pH, Dissolved Oxygen, and Coliform levels—sourced from the National Water Monitoring Programme (NWMP). This will enable rapid identification of water sources suitable for drinking, bathing, or irrigation.

### 2. Business Problem & Value

Deteriorating water quality and delayed identification of contamination pose severe risks to public health and environmental sustainability. This project addresses the challenge by leveraging data analytics to uncover pollution patterns and predict quality standards accurately. The key value lies in providing actionable insights for public health officials to prioritize interventions, monitor pollution trends like eutrophication or fecal contamination, and make informed decisions regarding water usage safety.

### 3. Scope & Methodology

Using a dataset of environmental features (e.g., Turbidity, Nitrate Nitrogen, BOD) and quality class labels, we will execute a complete data science workflow. This includes:

- **Exploratory Data Analysis (EDA):** To visualize correlations, detect outliers, and understand the distribution of water quality parameters.
- **Data Pre-processing:** To handle missing values, correct class imbalances, and engineer features relevant to environmental standards.
- **Model Development:** To train and validate multiple multiclass classification algorithms (e.g., Logistic Regression, Random Forest) to ensure high precision and recall.
- **Deployment (Optional):** To create an interactive Streamlit dashboard that allows users to input water metrics and receive instant quality classifications.

### 4. Expected Deliverable

The final deliverable will be a GitHub repository containing a robust analysis pipeline, trained model files, and a comprehensive report detailing model performance and key findings. This will be accompanied by an executive summary and presentation slides offering strategic recommendations for water quality monitoring and improvement.

## ✓ GitHub Link-

<https://github.com/manojDOX/Water-Quality-Classification-ML/tree/main>

## ✓ Problem Statement

The challenge is to classify water bodies in Maharashtra into specific quality categories (such as safe for drinking, bathing, or irrigation) based on their chemical and biological makeup. By analyzing parameters like pH, dissolved oxygen, and pollution levels, this project aims to build a machine learning model that automates water safety assessments to support public health decisions.

## ✓ 1. Know Your Data

## ✓ Import Libraries

```
1 # Core Libraries
2 import numpy as np
3 import pandas as pd
4
5 # Visualization
6 import matplotlib.pyplot as plt
7
8 # Preprocessing
9 from sklearn.preprocessing import StandardScaler
10
```

```
11 # Model Selection & Evaluation
12 from sklearn.model_selection import train_test_split, RandomizedSearchCV
13 from sklearn.metrics import (
14     confusion_matrix,
15     ConfusionMatrixDisplay,
16     f1_score
17 )
18
19 # Models
20 from sklearn.ensemble import RandomForestClassifier
21 from sklearn.linear_model import LogisticRegression
22 from sklearn.svm import SVC
23 from lightgbm import LGBMClassifier
24 from xgboost import XGBClassifier
25
26 # Statistics
27 from statsmodels.stats.outliers_influence import variance_inflation_factor
28
29 # Model Saving
30 import joblib
31
```

Dataset Loading

```
1 import pandas as pd
2
3 # Direct links to Google Drive CSVs
4 url1 = "https://drive.google.com/uc?id=15PWGmojzt9yzAtC24Hoppuakf33EP70C"
5 url2 = "https://drive.google.com/uc?id=1mp0_PrmbrfZtdVQxsl8Hqr_oQiahyaEL"
6 url3 = "https://drive.google.com/uc?id=1fSFAztPbZnrRdl3gFFaVgj3ItyRzkagU"
7
8 # Load each CSV as a DataFrame
9 df1 = pd.read_csv(url1, encoding="latin1")
10 df2 = pd.read_csv(url2, encoding="latin1")
11 df3 = pd.read_csv(url3, encoding="latin1")
12
13 # Optional: Print shapes to confirm
14 print("df1 shape:", df1.shape)
15 print("df2 shape:", df2.shape)
16 print("df3 shape:", df3.shape)
```

df1 shape: (222, 54)  
df2 shape: (222, 53)  
df3 shape: (222, 54)

Data First Look

```
1 df1.head()
```

|   | STN Code | Sampling Date | Month | Sampling Time | Stn Name                                          | Type Water Body | Name Of Water Body | River Basin | District           | State Name  | ... | Total Dissolved Solids | Total Fixed Solids | Total Suspended Solids | Phosphate | Boron    | Potass |
|---|----------|---------------|-------|---------------|---------------------------------------------------|-----------------|--------------------|-------------|--------------------|-------------|-----|------------------------|--------------------|------------------------|-----------|----------|--------|
| 0 | 1312     | 09-04-2025    | Sep   | 13:30:00      | Godavari river at Jaikwadi Dam, Village. Paith... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 448.0                  | 404.0              | 15                     | 0.4       | 0.5(BDL) |        |
| 1 | 2158     | 09-04-2025    | Sep   | 14:00:00      | Godavari river at U/s of Paithan at Paithan in... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 459.0                  | 414.0              | 14                     | 0.5       | 0.55     |        |
| 2 | 2159     | 09-04-2025    | Sep   | 14:30:00      | Godavari river at D/s of Paithan at Pathegaon ... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 449.0                  | 405.0              | 12                     | 0.62      | 0.51     |        |
| 3 | 2160     | 09-04-2025    | Sep   | 12:15:00      | Godavari river at U/s of Aurangabad Reservoir,... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 451.0                  | 406.0              | 12                     | 0.29      | 0.57     |        |
| 4 | 2161     | 09-03-2025    | Sep   | 17:00:00      | Godavari river at Jalna Intake water pump hous... | River           | Godavari           | Godavari    | Jalna              | Maharashtra | ... | 284.0                  | 256.0              | 11                     | 0.4       | 0.51     |        |

5 rows × 54 columns

```
1 df2.head()
```

|                     | STN Code | Sampling Date | Month | Sampling Time | Stn Name                                          | Type Water Body | Name Of Water Body | River Basin | District           | State Name  | ... | Sodium | Total Dissolved Solids | Total Fixed Solids | Total Suspended Solids | Phosphate | Boron |
|---------------------|----------|---------------|-------|---------------|---------------------------------------------------|-----------------|--------------------|-------------|--------------------|-------------|-----|--------|------------------------|--------------------|------------------------|-----------|-------|
| 0                   | 1312     | 07-03-2025    | Jul   | 14:45:00      | Godavari river at Jaikwadi Dam, Village. Paith... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 59.8   | 496.0                  | 447.0              | 10(BDL)                | 0.82      | 0.51  |
| 1                   | 2158     | 07-03-2025    | Jul   | 14:00:00      | Godavari river at U/s of Paithan at Paithan in... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 63.43  | 493.0                  | 444.0              | 10(BDL)                | 0.98      | 0.54  |
| 2                   | 2159     | 07-03-2025    | Jul   | 16:00:00      | Godavari river at D/s of Paithan at Pathegaon ... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 59.78  | 509.0                  | 459.0              | 11                     | 0.9       | 0.53  |
| 3                   | 2160     | 07-03-2025    | Jul   | 13:00:00      | Godavari river at U/s of Aurangabad Reservoir,... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 61.75  | 515.0                  | 464.0              | 11                     | 0.87      | 0.56  |
| 4                   | 2161     | 07-09-2025    | Jul   | 14:15:00      | Godavari river at Jalna Intake water pump hous... | River           | Godavari           | Godavari    | Jalna              | Maharashtra | ... | 111.8  | 806.0                  | 726.0              | 19                     | 0.8       | 0.56  |
| 5 rows × 53 columns |          |               |       |               |                                                   |                 |                    |             |                    |             |     |        |                        |                    |                        |           |       |

1 df3.head()

|                     | STN Code | Sampling Date | Month | Sampling Time | Stn Name                                          | Type Water Body | Name Of Water Body | River Basin | District           | State Name  | ... | Total Dissolved Solids | Total Fixed Solids | Total Suspended Solids | Phosphate | Boron | Potassium |
|---------------------|----------|---------------|-------|---------------|---------------------------------------------------|-----------------|--------------------|-------------|--------------------|-------------|-----|------------------------|--------------------|------------------------|-----------|-------|-----------|
| 0                   | 1312     | 08-05-2025    | Aug   | 16:30:00      | Godavari river at Jaikwadi Dam, Village. Paith... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 497.0                  | 448.0              | 10(BDL)                | 0.78      | 0.58  | 2.8       |
| 1                   | 2158     | 08-05-2025    | Aug   | 16:00:00      | Godavari river at U/s of Paithan at Paithan in... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 489.0                  | 441.0              | 11                     | 0.71      | 0.61  | 3.1       |
| 2                   | 2159     | 08-05-2025    | Aug   | 17:15:00      | Godavari river at D/s of Paithan at Pathegaon ... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 491.0                  | 442.0              | 11                     | 0.76      | 0.56  | 3.1       |
| 3                   | 2160     | 08-05-2025    | Aug   | 15:00:00      | Godavari river at U/s of Aurangabad Reservoir,... | River           | Godavari           | Godavari    | Ch. Sambhaji Nagar | Maharashtra | ... | 479.0                  | 612.0              | 10(BDL)                | 1.47      | 0.55  | 3.1       |
| 4                   | 2161     | 08-04-2025    | Aug   | 15:30:00      | Godavari river at Jalna Intake water pump hous... | River           | Godavari           | Godavari    | Jalna              | Maharashtra | ... | 765.0                  | 689.0              | 12                     | 0.33      | 0.56  | 4.5       |
| 5 rows × 54 columns |          |               |       |               |                                                   |                 |                    |             |                    |             |     |                        |                    |                        |           |       |           |

## ▾ Preparing main DataFrame

```

1 '''
2 Purpose:
3     Combine multiple monthly DataFrames into a single DataFrame while aligning all columns
4     and filling missing values with NaN.
5
6 Args:
7     df1 (DataFrame): First month data
8     df2 (DataFrame): Second month data
9     df3 (DataFrame): Third month data
10
11 Return:
12     combined_df (DataFrame): Merged DataFrame containing all columns from all inputs
13 '''

```

```

14 def combine_monthly_dfs(df1, df2, df3):
15     import pandas as pd
16
17     all_columns = set(df1.columns) | set(df2.columns) | set(df3.columns)
18
19     df1_aligned = df1.reindex(columns=all_columns)
20     df2_aligned = df2.reindex(columns=all_columns)
21     df3_aligned = df3.reindex(columns=all_columns)
22
23     combined_df = pd.concat([df1_aligned, df2_aligned, df3_aligned], ignore_index=True)
24
25     return combined_df
26
27
28 combined_df = combine_monthly_dfs(df1, df2, df3)
29

```

```
1 combined_df.info()
```

```

---
0   Approx Depth      645 non-null   object
1   Type Water Body   666 non-null   object
2   latitude          648 non-null   object
3   Name Of Water Body 600 non-null   object
4   Odor              164 non-null   object
5   Calcium CaCo3     645 non-null   float64
6   Use Based Class   645 non-null   object
7   Remark            8 non-null     object
8   Sampling Date     666 non-null   object
9   STN Code          666 non-null   int64
10  State Name        666 non-null   object
11  Total Kjeldahl N   644 non-null   object
12  Mon Agency        666 non-null   object
13  Potassium         644 non-null   object
14  Flow              640 non-null   float64
15  Major Polluting Sources 454 non-null   object
16  Total Fixed Solids 645 non-null   float64
17  BOD               645 non-null   object
18  Sampling Time     666 non-null   object
19  Weather           645 non-null   object
20  Color             645 non-null   object
21  Total Suspended Solids 645 non-null   object
22  Frequency         666 non-null   object
23  Amonia N          641 non-null   float64
24  District          666 non-null   object
25  Visibility Effluent Discharge 392 non-null   object
26  Use of Water in Down Stream 0 non-null     float64
27  Hardness CaCo3    645 non-null   float64
28  Floating Matter    645 non-null   object
29  longititude       648 non-null   object
30  River Basin       594 non-null   object
31  Phenophelene Alkanity 640 non-null   object
32  Nitrate N         643 non-null   object
33  Magnesium CaCo3   645 non-null   float64
34  Sulphate          645 non-null   object
35  Stn Name          666 non-null   object
36  Boron             567 non-null   object
37  Fecal Coliform    645 non-null   object
38  COD              645 non-null   float64
39  Fecal Streptococci 518 non-null   object
40  Dissolved O2      645 non-null   object
41  pH               645 non-null   float64
42  Sodium           645 non-null   object
43  Total Dissolved Solids 645 non-null   float64
44  Month            666 non-null   object
45  Total Coliform    645 non-null   object
46  Flouride         628 non-null   object
47  Nitrite N         0 non-null     float64
48  Conductivity      645 non-null   float64
49  Human Activities  645 non-null   object
50  Total Alkalinity  645 non-null   object
51  Temperature       583 non-null   float64
52  Turbidity         645 non-null   object
53  Chlorides         645 non-null   object
54  Phosphate         645 non-null   object
dtypes: float64(13), int64(1), object(41)
memory usage: 286.3+ KB

```

## Columns Info

1. **Phosphate** – Amount of phosphate nutrients present in water
2. **Floating Matter** – Presence of visible floating particles on water surface
3. **Nitrite N** – Nitrite nitrogen concentration (indicator of nitrogen pollution)
4. **Total Kjeldahl N** – Total organic nitrogen and ammonia nitrogen
5. **Temperature** – Water temperature at sampling time
6. **Visibility Effluent Discharge** – Whether effluent discharge is visible at site
7. **Fecal Streptococci** – Indicator bacteria showing fecal contamination
8. **Magnesium CaCo3** – Magnesium hardness expressed as CaCO3
9. **pH** – Acidity or alkalinity of water
10. **Major Polluting Sources** – Main pollution source near sampling location

11. **Calcium CaCo3** – Calcium hardness expressed as CaCO3
12. **Nitrate N** – Nitrate nitrogen concentration (fertilizer runoff indicator)
13. **Turbidity** – Cloudiness of water due to suspended particles
14. **Month** – Month when sample was collected
15. **Sampling Time** – Time of day sample was collected
16. **latitude** – Geographic latitude coordinate
17. **Name Of Water Body** – Name of river/lake/stream etc.
18. **Type Water Body** – Category of water body (River, Lake, Creek, etc.)
19. **Color** – Visual color of water
20. **Phenophelene Alkanity** – Alkalinity measured using phenolphthalein indicator
21. **Sulphate** – Sulphate ion concentration
22. **Dissolved O2** – Dissolved oxygen available for aquatic life
23. **Remark** – Additional comments about sample
24. **Mon Agency** – Monitoring agency name
25. **Use Based Class** – Water quality classification based on intended use
26. **Potassium** – Potassium ion concentration
27. **Sodium** – Sodium ion concentration
28. **Stn Name** – Monitoring station name
29. **Sampling Date** – Date of sample collection
30. **Flow** – Water flow rate
31. **Total Coliform** – Total coliform bacteria count
32. **Approx Depth** – Approximate water depth at site
33. **Fecal Coliform** – Fecal coliform bacteria count
34. **Use of Water in Down Stream** – How downstream water is used
35. **BOD** – Biochemical Oxygen Demand (organic pollution indicator)
36. **Boron** – Boron concentration
37. **District** – Administrative district name
38. **longitude** – Geographic longitude coordinate
39. **River Basin** – Basin where river belongs
40. **State Name** – State of sampling location
41. **Frequency** – Sampling frequency
42. **Total Alkalinity** – Water's capacity to neutralize acids
43. **COD** – Chemical Oxygen Demand (total organic matter)
44. **Conductivity** – Ability of water to conduct electricity (salinity)
45. **Odor** – Smell of water
46. **Human Activities** – Human activities near sampling site
47. **Weather** – Weather condition during sampling
48. **STN Code** – Station identification code
49. **Amonia N** – Ammonia nitrogen concentration
50. **Hardness CaCo3** – Total hardness expressed as CaCO3
51. **Total Dissolved Solids** – Total dissolved minerals and salts
52. **Flouride** – Fluoride concentration
53. **Chlorides** – Chloride ion concentration
54. **Total Suspended Solids** – Suspended particles in water
55. **Total Fixed Solids** – Inorganic solids remaining after heating

```

1 '''
2 Purpose:
3     Print all column names in numbered order.
4
5 Args:
6     df (DataFrame): Input DataFrame
7
8 Return:
9     None
10 '''
11 def print_numbered_columns(df):
12     for i, col in enumerate(df.columns, start=1):
13         print(f"{i}. {col}")
14
15
16 print_numbered_columns(combined_df)
17

```

1. Approx Depth
2. Type Water Body
3. latitude
4. Name Of Water Body
5. Odor
6. Calcium CaCo3
7. Use Based Class
8. Remark
9. Sampling Date
10. STN Code
11. State Name
12. Total Kjeldahl N
13. Mon Agency

```
14. Potassium
15. Flow
16. Major Polluting Sources
17. Total Fixed Solids
18. BOD
19. Sampling Time
20. Weather
21. Color
22. Total Suspended Solids
23. Frequency
24. Amonia N
25. District
26. Visibility Effluent Discharge
27. Use of Water in Down Stream
28. Hardness CaCo3
29. Floating Matter
30. longitude
31. River Basin
32. Phenophelene Alkanity
33. Nitrate N
34. Magnesium CaCo3
35. Sulphate
36. Stn Name
37. Boron
38. Fecal Coliform
39. COD
40. Fecal Streptococci
41. Dissolved O2
42. pH
43. Sodium
44. Total Dissolved Solids
45. Month
46. Total Coliform
47. Flouride
48. Nitrite N
49. Conductivity
50. Human Activities
51. Total Alkalinity
52. Temperature
53. Turbidity
54. Chlorides
55. Phosphate
```

## ▼ Data Wrangling

### ▼ Mising Values and Operation on values

```
1 '''
2 Purpose:
3     Remove rows where the specified column contains the value "No Information".
4
5 Args:
6     df (DataFrame): Input DataFrame
7     column (str): Column name to check
8
9 Return:
10    updated_df (DataFrame): DataFrame after removing matching rows
11 '''
12 def remove_no_information_rows(df, column):
13     updated_df = df[df[column] != "No Information"]
14     return updated_df
15
16
17 combined_df = remove_no_information_rows(combined_df, "Use Based Class")
18
```

```
1 '''
2 Purpose:
3     Print unique values for each column in the given DataFrame.
4
5 Args:
6     df (DataFrame): Input DataFrame
7
8 Return:
9     None
10 '''
11 def print_unique_values_per_column(df):
12     for col in df.columns:
13         print(f"\nColumn: {col}")
14         print(df[col].dropna().unique())
15
16
17 print_unique_values_per_column(combined_df)
18
```

Column: Chlorides

```
[ '53.98' '55.98' '26.99' '13' '12' '41.99' '39.99' '54.98' '43.99' '32.99'
'28.99' '29.99' '27.29' '17.14' '19.14' '18.69' '31.79' '11' '7' '7.85'
'12.85' '13.35' '18019.41' '17969.43' '207.14' '21.44' '7.8' '10.15'
'12.55' '11.35' '9.5' '8.5' '243.92' '227.93' '57.98' '73.48' '402.38'
'32.49' '19.49' '269.92' '11.5' '22.99' '335.9' '377.88' '357.89'
'10446.76' '12096.25' '85.97' '9547.04' '342.39' '404.87' '225.93'
'461.86' '13.5' '644.8' '539.83' '17.49' '10' '9' '215.93' '245.92' '8'
'10.5' '16.99' '10796.65' '15.5' '14' '30.49' '40.99' '154.95' '37.99'
'41.49' '17.99' '19.99' '20.99' '20.49' '6.5' '120.96' '31.49' '18.49'
'16.49' '23.49' '15' '16' '10.45' '14.2' '18.34' '10.6' '14.5' '18.89'
'16.44' '99.97' '31.14' '18.24' '20.29' '36.59' '35.54' '5(BDL)' '16.39'
'10.3' '6.65' '10.4' '14.8' '7.95' '6.75' '22.24' '9.9' '78.78' '51.93'
'60.58' '61.98' '47.19' '46.79' '65.48' '66.48' '64.98' '65.98' '112.96' '13.0' '38.99'
75.98 66.98 63.98 18.99 33.99 15.25 18.89 17.09 13.8 12.45 9.5 10.25
11.85 20.04 11.25 13495.82 14320.56 2764.14 37.04 13.25 13.6 11.1 6.54
7.5 47.32 45.31 44.3 20.64 19.63 20.14 2391.1 11.07 1711.53 3121.02
10973.9 10369.83 12596.09 17394.61 4102.63 13695.75 1208.14 221.49 193.3
387.61 8.56 14.09 14.6 458.08 8.05 13.59 9.06 1459.83 289.95 293.98 9.56
10.07 26.49 167.95 5.5 7.0 64.48 97.97 57.48 44.99 49.98 72.48 80.97
1314.59 24.99 29.99 14.5 15.5 46.49 22.99 19.99 16.99 15.0 39.99 34.99
34.49 17.49 19.49 30.99 17.99 18.49 11.5 16.0 131.96 14.0 23.99 27.49
10.5 12.5 12.0 57.98 99.97 46.99 50.48 8.5 11.05 16.94 20.34 15.15 15.8
18.44 18.29 27.89 15.45 15.1 16.44 18.19 28.89 41.49 79.18 15.75 7.4
21.19 7.25 19.84 29.94 10.65 9.75 9.0 9.7 7.85 55.18 133.16 65.08 73.43
'58.48' '63.48' '58.98' '98.47' '52.48' '50.48' '18.99' '24.39' '8.35'
'17.74' '18.54' '15.25' '6.9' '150.15' '26.39' '18069.4' '18344.31'
'10246.82' '18.59' '6.7' '8.85' '8.8' '72.98' '71.48' '74.98' '3673.86'
'3623.88' '3498.92' '3523.91' '12296.19' '8797.27' '11996.28' '16194.98'
'15395.23' '12696.06' '13495.82' '130.96' '13195.91' '1109.66' '709.78'
'279.91' '1909.41' '3598.88' '77.98' '76.98' '75.98' '4448.62' '485.85'
'471.85' '4773.52' '103.62' '11.8' '27.14' '25.44' '22.49' '41.04'
'57.73' '81.47' '1044.68' '65.98' '25.99' '38.49' '64.98' '69.98' '46.99'
'100.97' '9.3' '11.2' '7.45' '26.49' '27.99' '27.49' '28.49' '113.96'
'46.49' '24.49' '25.49' '15.15' '44.89' '41.74' '25.59' '56.68' '33.29'
'74.18' '39.69' '44.44' '44.49' '41.24' '32.29' '12.8' '12.9' '7.9'
'15.95' '7.35' '21.74' '10.55' '67.43' '62.23' '82.42' '78.93' '59.48']
```

Column: Phosphate

```
[ '0.4' '0.5' '0.62' '0.29' '0.21' '0.77' '0.32' '0.27' '0.31' '0.2(BDL)'
'ND' '2.8' '0.22' '0.57' '0.44' '0.49' '0.9' '0.53' '1.14' '0.61' '1.5'
'0.8' '1.43' '0.59' '0.86' '0.84' '1.45' '1.42' '1.15' '0.81' '0.67'
'0.63' '0.68' '1.03' '1.07' '0.89' '1.05' '1.01' '0.66' '1' '0.88' '0.73'
'0.45' '0.82' '0.98' '0.87' '0.99' '1.84' '0.97' '0.38' '0.26' '0.37'
'0.3' '0.39' '0.23' '0.24' '0.25' '0.43' '0.35' '1.55' '0.28' '0.33'
'0.74' '1.08' '0.69' '1.49' '0.56' '0.55' '0.79' '0.78' '0.96' '1.4'
'0.47' '1.04' '1.31' '1.1' '1.25' '0.93' '1.18' '0.46' '0.51' '0.71'
'0.76' '1.47' '1.12' '1.02' '0.34' '2' '0.36' '1.36' '1.11' '1.73' '1.81'
'1.13' '0.75' '0.94' '5.2' '0.91' '3.68' '2.16' '1.41' '0.72' '0.95'
'0.83' '0.48']
```

```
1 '''
2 Purpose:
3     Print each column name with its missing value percentage in aligned format.
4
5 Args:
6     df (DataFrame): Input DataFrame
7
8 Return:
9     None
10 '''
11 def print_missing_percentage(df):
12     total_rows = len(df)
13
14     for col in df.columns:
15         missing_percent = (df[col].isna().sum() / total_rows) * 100
16         print(f"{col:<35} {missing_percent:>7.2f}%")
17
18
19 print_missing_percentage(combined_df)
20
```

|                         |        |
|-------------------------|--------|
| Approx Depth            | 3.95%  |
| Type Water Body         | 0.00%  |
| Latitude                | 2.44%  |
| Name Of Water Body      | 10.34% |
| Odor                    | 80.26% |
| Calcium CaCo3           | 3.95%  |
| Use Based Class         | 3.95%  |
| Remark                  | 98.50% |
| Sampling Date           | 0.00%  |
| STN Code                | 0.00%  |
| State Name              | 0.00%  |
| Total Kjeldahl N        | 4.14%  |
| Mon Agency              | 0.00%  |
| Potassium               | 4.14%  |
| Flow                    | 4.89%  |
| Major Polluting Sources | 23.12% |
| Total Fixed Solids      | 3.95%  |
| BOD                     | 3.95%  |
| Sampling Time           | 0.00%  |
| Weather                 | 3.95%  |
| Color                   | 3.95%  |
| Total Suspended Solids  | 3.95%  |
| Frequency               | 0.00%  |
| Amonia N                | 4.70%  |

|                               |         |
|-------------------------------|---------|
| District                      | 0.00%   |
| Visibility Effluent Discharge | 37.22%  |
| Use of Water in Down Stream   | 100.00% |
| Hardness CaCo3                | 3.95%   |
| Floating Matter               | 3.95%   |
| longitude                     | 2.44%   |
| River Basin                   | 11.47%  |
| Phenophelene Alkanity         | 4.70%   |
| Nitrate N                     | 4.32%   |
| Magnesium CaCo3               | 3.95%   |
| Sulphate                      | 3.95%   |
| Stn Name                      | 0.00%   |
| Boron                         | 13.91%  |
| Fecal Coliform                | 3.95%   |
| COD                           | 3.95%   |
| Fecal Streptococci            | 21.43%  |
| Dissolved O2                  | 3.95%   |
| pH                            | 3.95%   |
| Sodium                        | 3.95%   |
| Total Dissolved Solids        | 3.95%   |
| Month                         | 0.00%   |
| Total Coliform                | 3.95%   |
| Flouride                      | 7.14%   |
| Nitrite N                     | 100.00% |
| Conductivity                  | 3.95%   |
| Human Activities              | 3.95%   |
| Total Alkalinity              | 3.95%   |
| Temperature                   | 15.60%  |
| Turbidity                     | 3.95%   |
| Chlorides                     | 3.95%   |
| Phosphate                     | 3.95%   |

Remark : Null refer to no reason and values are intact but if Non Null then that means the water bodies are dried out and not able to perform any calibration

```

1 '''
2 Purpose:
3     Remove rows where the Remark column has non-null values and keep only rows
4     where Remark is null.
5
6 Args:
7     df (DataFrame): Input DataFrame
8
9 Return:
10    cleaned_df (DataFrame): DataFrame containing only rows with null Remark
11 '''
12 def remove_non_null_remark_rows(df):
13     cleaned_df = df[df["Remark"].isna()]
14
15     return cleaned_df
16
17
18 combined_df = remove_non_null_remark_rows(combined_df)
19 print_missing_percentage(combined_df)

```

|                               |         |
|-------------------------------|---------|
| Approx Depth                  | 2.48%   |
| Type Water Body               | 0.00%   |
| latitude                      | 2.48%   |
| Name Of Water Body            | 10.50%  |
| Odor                          | 79.96%  |
| Calcium CaCo3                 | 2.48%   |
| Use Based Class               | 2.48%   |
| Remark                        | 100.00% |
| Sampling Date                 | 0.00%   |
| STN Code                      | 0.00%   |
| State Name                    | 0.00%   |
| Total Kjeldahl N              | 2.67%   |
| Mon Agency                    | 0.00%   |
| Potassium                     | 2.67%   |
| Flow                          | 3.44%   |
| Major Polluting Sources       | 21.95%  |
| Total Fixed Solids            | 2.48%   |
| BOD                           | 2.48%   |
| Sampling Time                 | 0.00%   |
| Weather                       | 2.48%   |
| Color                         | 2.48%   |
| Total Suspended Solids        | 2.48%   |
| Frequency                     | 0.00%   |
| Amonia N                      | 3.24%   |
| District                      | 0.00%   |
| Visibility Effluent Discharge | 36.26%  |
| Use of Water in Down Stream   | 100.00% |
| Hardness CaCo3                | 2.48%   |
| Floating Matter               | 2.48%   |
| longitude                     | 2.48%   |
| River Basin                   | 11.64%  |
| Phenophelene Alkanity         | 3.24%   |
| Nitrate N                     | 2.86%   |
| Magnesium CaCo3               | 2.48%   |
| Sulphate                      | 2.48%   |
| Stn Name                      | 0.00%   |
| Boron                         | 12.60%  |
| Fecal Coliform                | 2.48%   |
| COD                           | 2.48%   |
| Fecal Streptococci            | 20.23%  |
| Dissolved O2                  | 2.48%   |

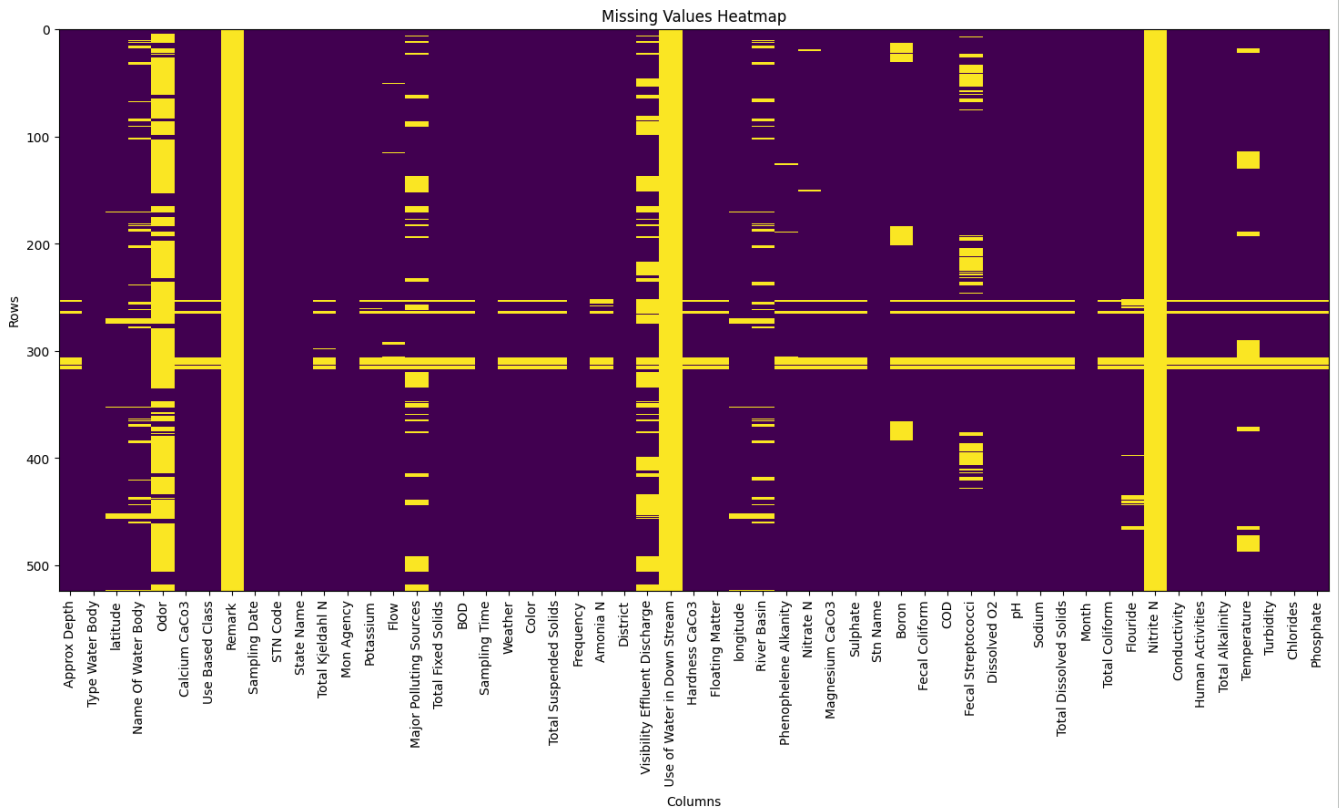


|                        |         |
|------------------------|---------|
| pH                     | 2.48%   |
| Sodium                 | 2.48%   |
| Total Dissolved Solids | 2.48%   |
| Month                  | 0.00%   |
| Total Coliform         | 2.48%   |
| Flouride               | 5.73%   |
| Nitrite N              | 100.00% |
| Conductivity           | 2.48%   |
| Human Activities       | 2.48%   |
| Total Alkalinity       | 2.48%   |
| Temperature            | 14.31%  |
| Turbidity              | 2.48%   |
| Chlorides              | 2.48%   |
| Phosphate              | 2.48%   |

```

1 '''
2 Purpose:
3     Plot a heatmap showing missing values with clear cell boundaries,
4     rows representing records and columns representing features.
5
6 Args:
7     df (DataFrame): Input DataFrame
8
9 Return:
10    None
11 '''
12 def plot_missing_values_heatmap(df):
13     import matplotlib.pyplot as plt
14     import numpy as np
15
16     missing_matrix = df.isna().astype(int)
17
18     plt.figure(figsize=(18, 8))
19     plt.imshow(missing_matrix, aspect="auto", interpolation="nearest")
20     plt.xticks(range(len(df.columns)), df.columns, rotation=90)
21     plt.xlabel("Columns")
22     plt.ylabel("Rows")
23     plt.title("Missing Values Heatmap")
24     plt.grid(False)
25     plt.show()
26
27
28 plot_missing_values_heatmap(combined_df)
29

```



```

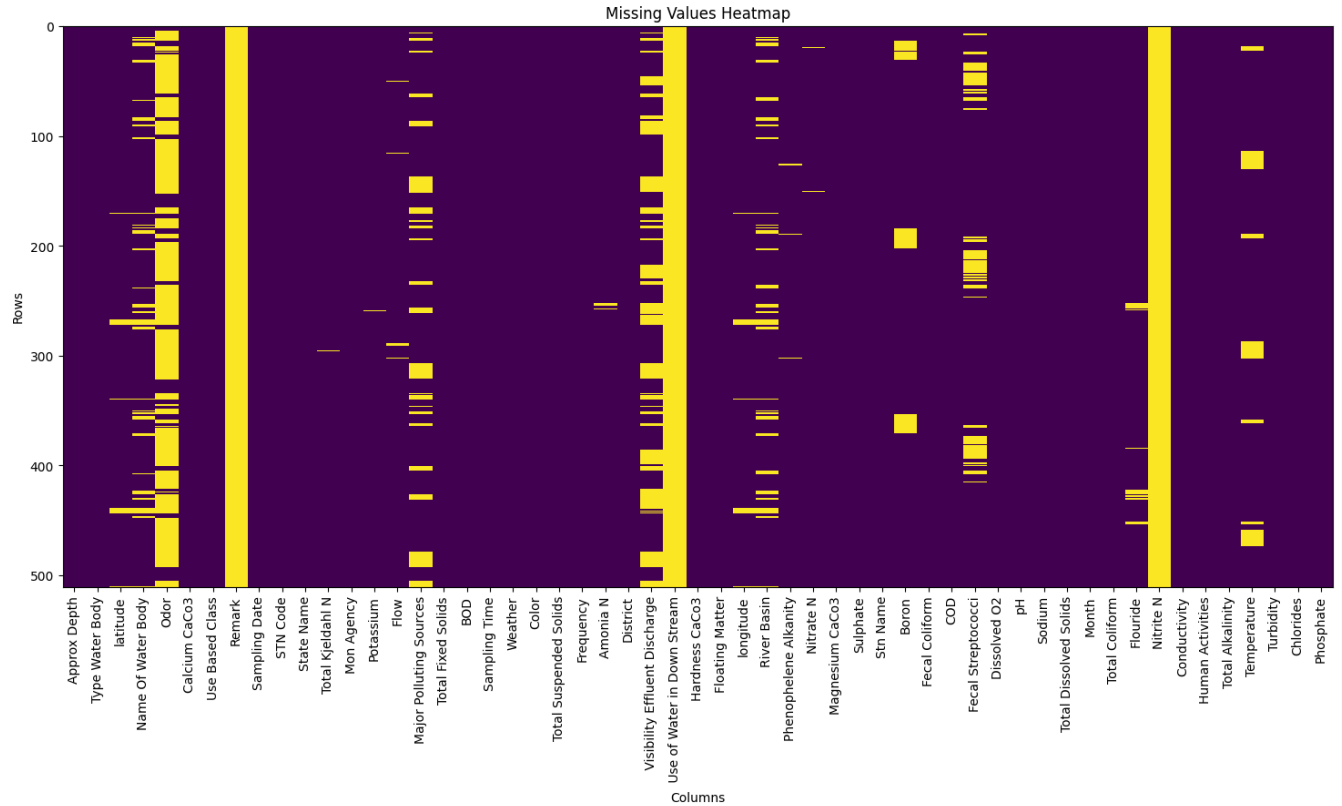
1 '''
2 Purpose:
3     Remove rows that contain null values in any of the specified columns.
4
5 Args:
6     df (DataFrame): Input DataFrame

```

```

7     columns (list): List of column names to check for null values
8
9 Return:
10     cleaned_df (DataFrame): DataFrame with rows removed where null exists in given columns
11 '''
12 def remove_rows_with_nulls_in_columns(df, columns):
13     cleaned_df = df.dropna(subset=columns)
14     return cleaned_df
15
16
17 combined_df = remove_rows_with_nulls_in_columns(combined_df, ["Calcium CaCo3"])
18 plot_missing_values_heatmap(combined_df)
19

```

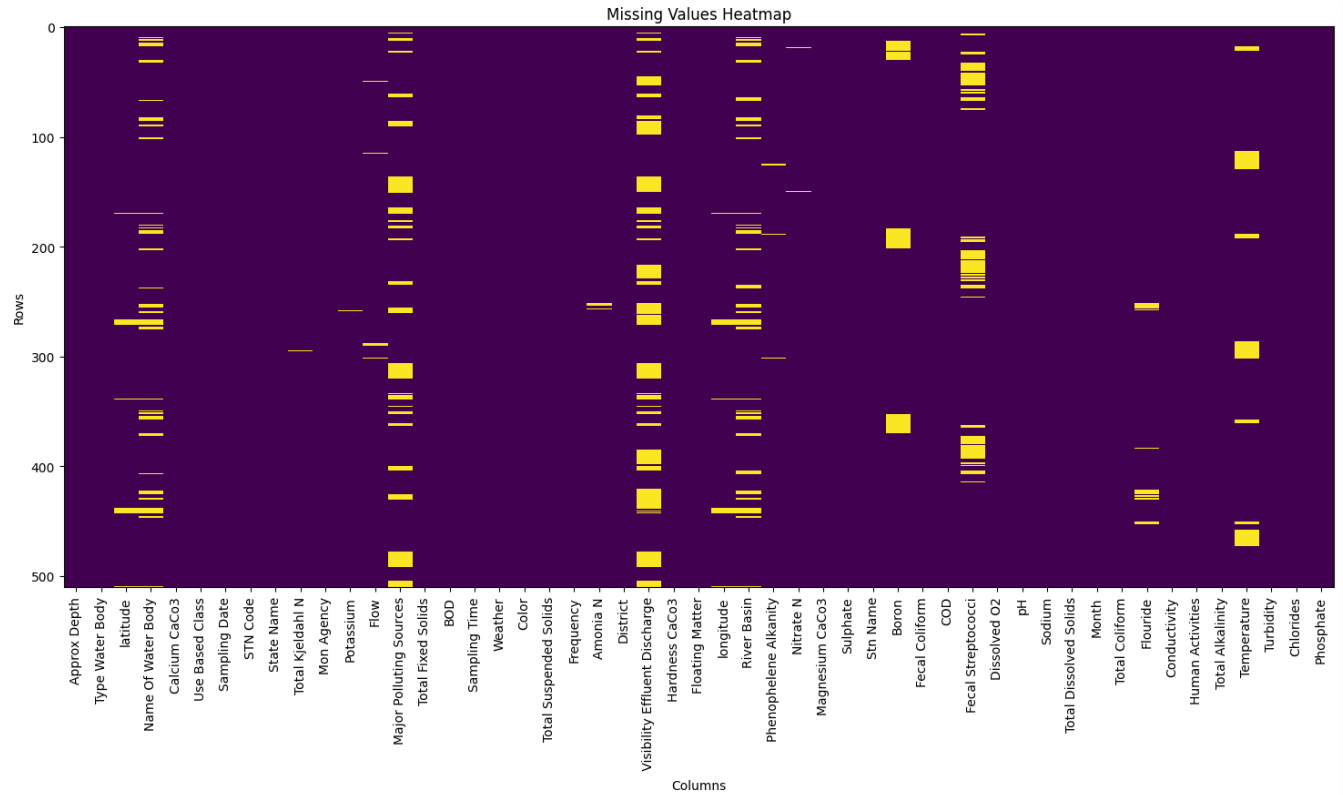


```

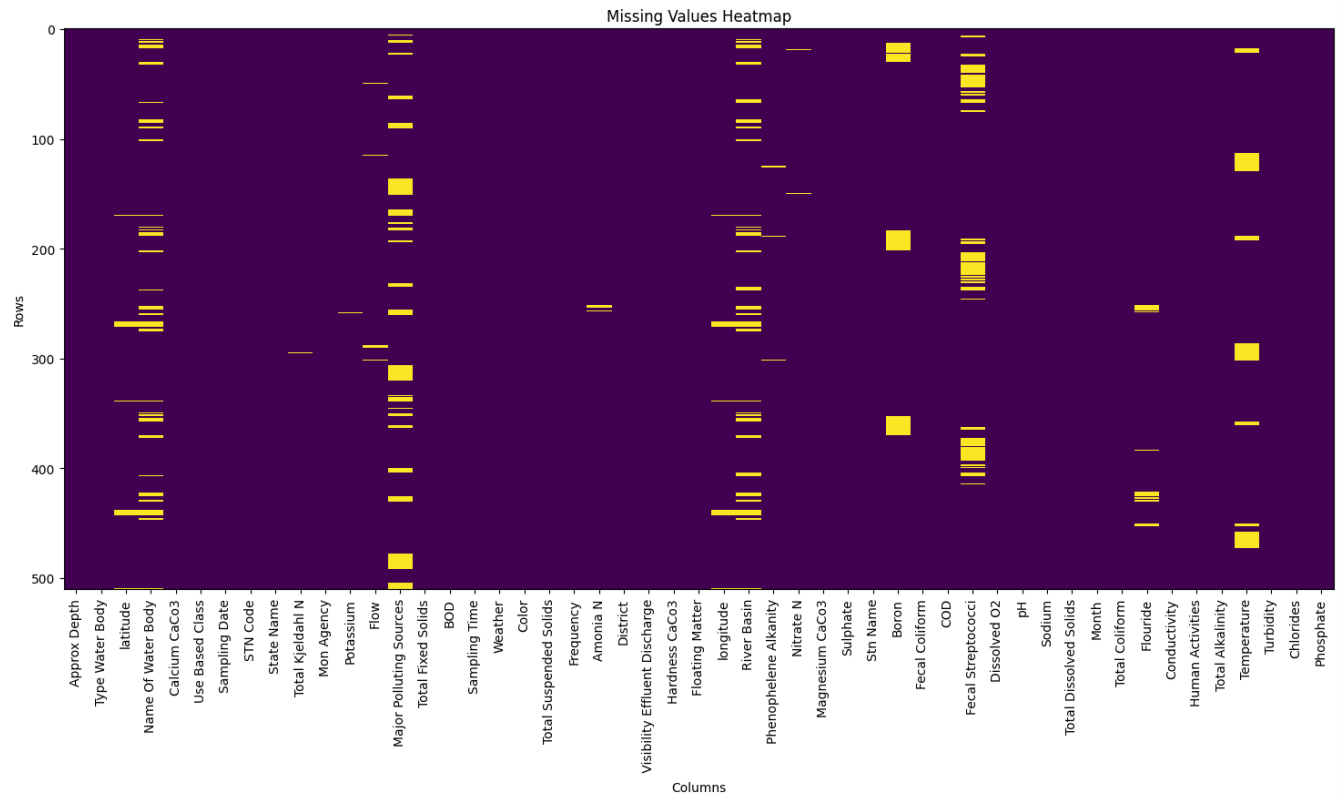
1 '''
2 Purpose:
3     Drop specified columns from the DataFrame.
4
5 Args:
6     df (DataFrame): Input DataFrame
7     columns (list): List of column names to be dropped
8
9 Return:
10     updated_df (DataFrame): DataFrame after dropping given columns
11 '''
12 def drop_columns(df, columns):
13     updated_df = df.drop(columns=columns, errors="ignore")
14     print("Following columns has been dropped:")
15     print(', '.join(columns))
16     return updated_df
17
18
19 combined_df = drop_columns(combined_df, ["Remark", "Nitrite N", " Use of Water in Down Stream", "Odor"])
20 plot_missing_values_heatmap(combined_df)

```

Following columns has been dropped:  
Remark, Nitrite N, Use of Water in Down Stream, Odor



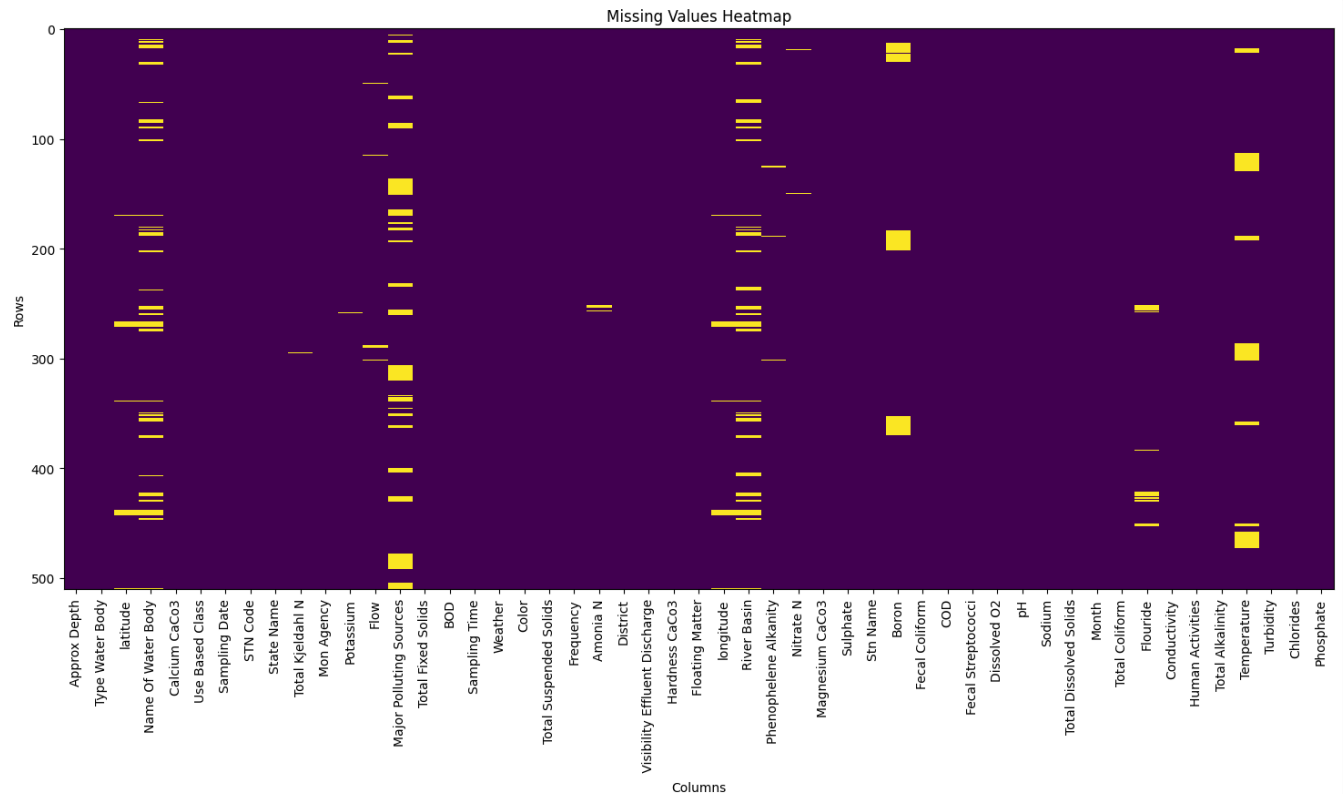
```
1 '''
2 Purpose:
3     Replace null values in a specified column with a given value.
4
5 Args:
6     df (DataFrame): Input DataFrame
7     column (str): Column name where null values should be filled
8     value: Value to replace null entries
9
10 Return:
11     updated_df (DataFrame): DataFrame with nulls replaced in given column
12 '''
13 def fill_null_with_value(df, column, value):
14     updated_df = df.copy()
15     updated_df[column] = updated_df[column].fillna(value)
16     return updated_df
17
18
19 combined_df = fill_null_with_value(combined_df, " Visibility Effluent Discharge", "No Discharge")
20 plot_missing_values_heatmap(combined_df)
```



```

1 '''
2 Purpose:
3     Replace multiple specific values in a given column using a mapping dictionary
4     and fill null values with a specified value.
5
6 Args:
7     df (DataFrame): Input DataFrame
8     column (str): Column name where replacements should occur
9     replace_dict (dict): Dictionary mapping old values to new values
10    fill_value: Value used to replace null entries
11
12 Return:
13    updated_df (DataFrame): DataFrame with values replaced and nulls filled
14 '''
15 def replace_values_and_fill_null(df, column, replace_dict, fill_value):
16     updated_df = df.copy()
17     updated_df[column] = updated_df[column].replace(replace_dict)
18     updated_df[column] = updated_df[column].fillna(fill_value)
19     return updated_df
20
21
22 combined_df = replace_values_and_fill_null(
23     combined_df,
24     "Fecal Streptococci",
25     {"1.8(BDL)": "1.8", "BDL": "1.5"},
26     "0"
27 )
28
29 plot_missing_values_heatmap(combined_df)

```



```
1 combined_df['Temperature'].unique()
```

```
array([28. , 25. , 24. , 19. , 18. , 32. , 22. , 23. , nan, 26. , 29. ,
       30. , 27. , 20. , 16. , 21. , 27.5, 26.5, 17. , 31. ])
```

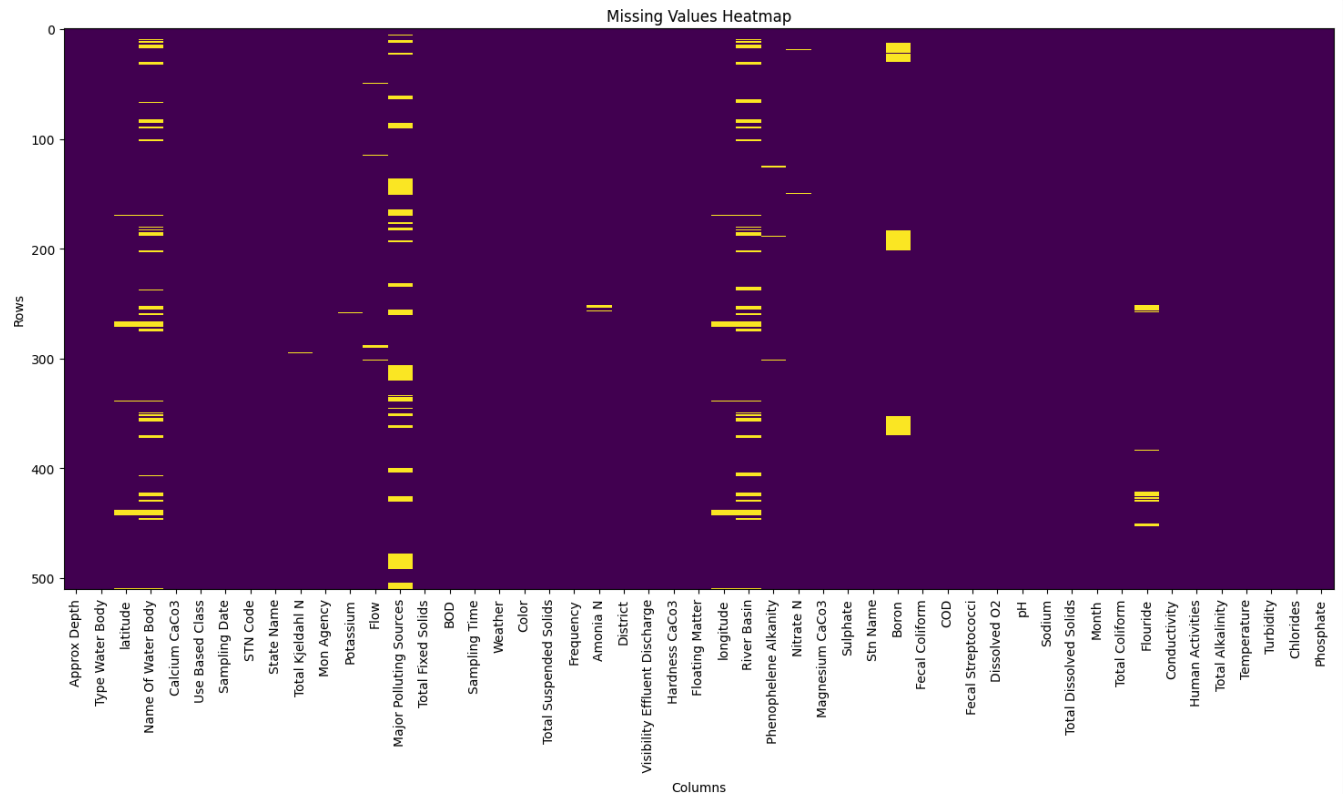
```
1 combined_df['Temperature'].mean()
```

```
np.float64(25.292873051224944)
```

```
1 combined_df = replace_values_and_fill_null(
2     combined_df,
3     "Temperature",
4     {}),
5     combined_df['Temperature'].mean()
6 )
```

```
1 '''
2 Purpose:
3     Convert all values of a specified column to float type.
4
5 Args:
6     df (DataFrame): Input DataFrame
7     column (str): Column name to be converted to float
8
9 Return:
10    updated_df (DataFrame): DataFrame with column converted to float
11 '''
12 def convert_column_to_float(df, column):
13     updated_df = df.copy()
14     updated_df[column] = updated_df[column].astype(float)
15     return updated_df
16
17
18 combined_df = convert_column_to_float(combined_df, "Temperature")
19
```

```
1 plot_missing_values_heatmap(combined_df)
```



```

1 # '''
2 # Purpose:
3 # Iterate through all columns, display unique values, ask user whether to convert
4 # column to float, convert numeric values while keeping string values unchanged,
5 # and record all actions performed.
6
7 # Args:
8 #     df (DataFrame): Input DataFrame
9
10 # Return:
11 #     updated_df (DataFrame): Updated DataFrame after user-driven conversions
12 # '''
13 def interactive_column_processing(df):
14     import pandas as pd
15
16     updated_df = df.copy()
17     actions_log = {}
18
19     for col in updated_df.columns:
20         print("\nColumn:", col)
21         print("Unique Values:", updated_df[col].dropna().unique())
22
23         choice = input("Convert this column to float? (yes/no): ").strip().lower()
24
25         if choice == "yes":
26             temp = pd.to_numeric(updated_df[col], errors="coerce")
27             updated_df[col] = temp.combine_first(updated_df[col])
28             actions_log[col] = "Converted numeric values to float (strings kept)"
29         else:
30             actions_log[col] = "Skipped"
31
32     print("\nSummary of Actions:")
33     for col, action in actions_log.items():
34         print(f"{col} -> {action}")
35
36     return updated_df
37
38
39 # combined_df = interactive_column_processing(combined_df)
40

```

```

1 # '''
2 # Purpose:
3 # Iterate through columns, display unique values, ask user to skip or operate.
4 # If operate, detect all unique values containing substring "BDL", ask user
5 # replacement for each, apply replacements, and record all actions performed.
6 # Finally print a detailed summary for future function creation.
7
8 # Args:

```

```

9 #     df (DataFrame): Input DataFrame
10
11 # Return:
12 #     updated_df (DataFrame): DataFrame after applying BDL replacements
13 # '''
14 # def interactive_bdl_processing(df):
15 #     updated_df = df.copy()
16 #     actions_log = {}
17
18 #     for col in updated_df.columns:
19 #         print("\nColumn:", col)
20 #         uniques = updated_df[col].dropna().unique()
21 #         print("Unique Values:", uniques)
22
23 #         choice = input("Skip or Operate? (skip: s/operate: o): ").strip().lower()
24
25 #         if choice == "o":
26 #             bdl_values = [val for val in uniques if "BDL" in str(val)]
27 #             col_actions = {}
28
29 #             for val in bdl_values:
30 #                 new_val = input(f'Value "{val}" contains BDL. Enter replacement: ')
31 #                 updated_df[col] = updated_df[col].replace(val, new_val)
32 #                 col_actions[val] = new_val
33
34 #             if col_actions:
35 #                 actions_log[col] = col_actions
36 #             else:
37 #                 actions_log[col] = "No BDL values found"
38 #         else:
39 #             actions_log[col] = "Skipped"
40
41 #     print("\nSummary of Actions:")
42 #     for col, action in actions_log.items():
43 #         print(f"{col} -> {action}")
44
45 #     return updated_df
46
47
48 # combined_df = interactive_bdl_processing(combined_df)
49

```

```

1 '''
2 Purpose:
3     Apply predefined BDL substring replacement mappings on columns based on
4     previously recorded user decisions and update the DataFrame automatically.
5
6 Args:
7     df (DataFrame): Input DataFrame
8
9 Return:
10    updated_df (DataFrame): DataFrame after applying BDL replacements
11 '''
12 def apply_bdl_replacements(df):
13     updated_df = df.copy()
14
15     bdl_mappings = {
16         "Phosphate": {"0.2(BDL)": "0.2"},
17         "Total Kjeldahl N": {"1.5(BDL)": "1.5"},
18         "Nitrate N": {"0.3(BDL)": "0.3"},
19         "Turbidity": {"1(BDL)": "1"},
20         "Phenophelene Alkanity": {"5(BDL)": "5", "BDL": "2.5"},
21         "Sulphate": {"5(BDL)": "5", "BDL": "2.5"},
22         "Dissolved O2": {"0.3(BDL)": "0.3"},
23         "Potassium": {"1(BDL)": "1", "BDL": "0.5"},
24         "Sodium": {"5(BDL)": "5"},
25         "Total Coliform": {"1.8(BDL)": "1.8"},
26         "Fecal Coliform": {"1.8(BDL)": "1.8"},
27         "BOD": {"1(BDL)": "1"},
28         "Boron": {"0.5(BDL)": "0.5"},
29         "Total Alkalinity": {"20(BDL)": "20"},
30         "Flouride": {"0.2(BDL)": "0.2", "BDL": "0.1"},
31         "Chlorides": {"5(BDL)": "5"},
32         "Total Suspended Solids": {"10(BDL)": "10", "BDL": "5"}
33     }
34
35     actions_log = {}
36
37     for col, mapping in bdl_mappings.items():
38         if col in updated_df.columns:
39             updated_df[col] = updated_df[col].replace(mapping)
40             actions_log[col] = mapping
41
42     print("\nSummary of Actions:")
43     for col, action in actions_log.items():
44         print(f"{col} -> {action}")
45
46     return updated_df
47
48

```

```
49 combined_df = apply_bdl_replacements(combined_df)
50
```

```
Summary of Actions:
Phosphate -> {'0.2(BDL)': '0.2'}
Total Kjeldahl N -> {'1.5(BDL)': '1.5'}
Nitrate N -> {'0.3(BDL)': '0.3'}
Turbidity -> {'1(BDL)': '1'}
Phenophelene Alkanity -> {'5(BDL)': '5', 'BDL': '2.5'}
Sulphate -> {'5(BDL)': '5', 'BDL': '2.5'}
Dissolved O2 -> {'0.3(BDL)': '0.3'}
Potassium -> {'1(BDL)': '1', 'BDL': '0.5'}
Sodium -> {'5(BDL)': '5'}
Total Coliform -> {'1.8(BDL)': '1.8'}
Fecal Coliform -> {'1.8(BDL)': '1.8'}
BOD -> {'1(BDL)': '1'}
Boron -> {'0.5(BDL)': '0.5'}
Total Alkalinity -> {'20(BDL)': '20'}
Flouride -> {'0.2(BDL)': '0.2', 'BDL': '0.1'}
Chlorides -> {'5(BDL)': '5'}
Total Suspended Solids -> {'10(BDL)': '10', 'BDL': '5'}
```

```
1 '''
2 Purpose:
3     Apply predefined conversion actions on columns based on previously
4     recorded user decisions and print summary of actions performed.
5
6 Args:
7     df (DataFrame): Input DataFrame
8
9 Return:
10    updated_df (DataFrame): DataFrame after applying recorded actions
11 '''
12 def apply_recorded_column_actions(df):
13     import pandas as pd
14
15     updated_df = df.copy()
16
17     convert_columns = [
18         "Phosphate",
19         "Total Kjeldahl N",
20         "Temperature",
21         "Fecal Streptococci",
22         "Magnesium CaCo3",
23         "pH",
24         "Calcium CaCo3",
25         "Nitrate N",
26         "Turbidity",
27         "Phenophelene Alkanity",
28         "Sulphate",
29         "Dissolved O2",
30         "Potassium",
31         "Sodium",
32         "Flow",
33         "Total Coliform",
34         "Fecal Coliform",
35         "BOD",
36         "Boron",
37         "Total Alkalinity",
38         "COD",
39         "Amonia N",
40         "Hardness CaCo3",
41         "Flouride",
42         "Chlorides",
43         "Total Suspended Solids"
44     ]
45
46     skipped_columns = [
47         "Floating Matter",
48         "Visibility Effluent Discharge",
49         "Major Polluting Sources",
50         "Month",
51         "Sampling Time",
52         "Latitude",
53         "Name Of Water Body",
54         "Type Water Body",
55         "Color",
56         "Mon Agency",
57         "Use Based Class",
58         "Stn Name",
59         "Sampling Date",
60         "Approx Depth",
61         "District",
62         "longitude",
63         "River Basin",
64         "State Name",
65         "Frequency",
66         "Conductivity",
67         "Human Activities",
68         "Weather",
69         "STN Code",
```



```

70     "Total Dissolved Solids",
71     "Total Fixed Solids"
72 ]
73
74 actions_log = {}
75
76 for col in convert_columns:
77     if col in updated_df.columns:
78         numeric_version = pd.to_numeric(updated_df[col], errors="coerce")
79         updated_df[col] = numeric_version.combine_first(updated_df[col])
80         actions_log[col] = "Converted numeric values to float (strings kept)"
81
82 for col in skipped_columns:
83     if col in updated_df.columns:
84         actions_log[col] = "Skipped"
85
86 print("\nSummary of Actions:")
87 for col, action in actions_log.items():
88     print(f"{col} -> {action}")
89
90 return updated_df
91
92
93 combined_df = apply_recorded_column_actions(combined_df)
94

```

```

Temperature -> Converted numeric values to float (strings kept)
Fecal Streptococci -> Converted numeric values to float (strings kept)
Magnesium CaCo3 -> Converted numeric values to float (strings kept)
pH -> Converted numeric values to float (strings kept)
Calcium CaCo3 -> Converted numeric values to float (strings kept)
Nitrate N -> Converted numeric values to float (strings kept)
Turbidity -> Converted numeric values to float (strings kept)
Phenophelene Alkanity -> Converted numeric values to float (strings kept)
Sulphate -> Converted numeric values to float (strings kept)
Dissolved O2 -> Converted numeric values to float (strings kept)
Potassium -> Converted numeric values to float (strings kept)
Sodium -> Converted numeric values to float (strings kept)
Flow -> Converted numeric values to float (strings kept)
Total Coliform -> Converted numeric values to float (strings kept)
Fecal Coliform -> Converted numeric values to float (strings kept)
BOD -> Converted numeric values to float (strings kept)
Boron -> Converted numeric values to float (strings kept)
Total Alkalinity -> Converted numeric values to float (strings kept)
COD -> Converted numeric values to float (strings kept)
Amonia N -> Converted numeric values to float (strings kept)
Hardness CaCo3 -> Converted numeric values to float (strings kept)
Flouride -> Converted numeric values to float (strings kept)
Chlorides -> Converted numeric values to float (strings kept)
Total Suspended Solids -> Converted numeric values to float (strings kept)
Floating Matter -> Skipped
Visibility Effluent Discharge -> Skipped
Major Polluting Sources -> Skipped
Month -> Skipped
Sampling Time -> Skipped
latitude -> Skipped
Name Of Water Body -> Skipped
Type Water Body -> Skipped
Color -> Skipped
Mon Agency -> Skipped
Use Based Class -> Skipped
Stn Name -> Skipped
Sampling Date -> Skipped
Approx Depth -> Skipped
District -> Skipped
longitude -> Skipped
River Basin -> Skipped
State Name -> Skipped
Frequency -> Skipped
Conductivity -> Skipped
Human Activities -> Skipped
Weather -> Skipped
STN Code -> Skipped
Total Dissolved Solids -> Skipped
Total Fixed Solids -> Skipped
/tmp/ipython-input-4258039563.py:79: FutureWarning: The behavior of array concatenation with empty entries is deprecated. In a future version, this will
  updated_df[col] = numeric_version.combine_first(updated_df[col])
/tmp/ipython-input-4258039563.py:79: FutureWarning: The behavior of array concatenation with empty entries is deprecated. In a future version, this will
  updated_df[col] = numeric_version.combine_first(updated_df[col])
/tmp/ipython-input-4258039563.py:79: FutureWarning: The behavior of array concatenation with empty entries is deprecated. In a future version, this will
  updated_df[col] = numeric_version.combine_first(updated_df[col])
/tmp/ipython-input-4258039563.py:79: FutureWarning: The behavior of array concatenation with empty entries is deprecated. In a future version, this will
  updated_df[col] = numeric_version.combine_first(updated_df[col])

```

```
1 combined_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 511 entries, 0 to 665
Data columns (total 51 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Approx Depth           511 non-null    object
1   Type Water Body        511 non-null    object
2   latitude               498 non-null    object
3   Name Of Water Body     456 non-null    object
4   Calcium CaCo3         511 non-null    float64

```

```

5 Use Based Class          511 non-null object
6 Sampling Date            511 non-null object
7 STN Code                 511 non-null int64
8 State Name              511 non-null object
9 Total Kjeldahl N        510 non-null object
10 Mon Agency              511 non-null object
11 Potassium               510 non-null object
12 Flow                   506 non-null float64
13 Major Polluting Sources 409 non-null object
14 Total Fixed Solids      511 non-null float64
15 BOD                    511 non-null float64
16 Sampling Time           511 non-null object
17 Weather                 511 non-null object
18 Color                  511 non-null object
19 Total Suspended Solids  511 non-null int64
20 Frequency               511 non-null object
21 Amonia N                507 non-null float64
22 District                511 non-null object
23 Visibility Effluent Discharge 511 non-null object
24 Hardness CaCo3          511 non-null float64
25 Floating Matter         511 non-null object
26 longitude               498 non-null object
27 River Basin             450 non-null object
28 Phenophelene Alkanity   507 non-null object
29 Nitrate N               509 non-null object
30 Magnesium CaCo3         511 non-null float64
31 Sulphate                511 non-null float64
32 Stn Name                511 non-null object
33 Boron                   458 non-null object
34 Fecal Coliform          511 non-null float64
35 COD                     511 non-null float64
36 Fecal Streptococci      511 non-null float64
37 Dissolved O2            511 non-null float64
38 pH                      511 non-null float64
39 Sodium                  511 non-null float64
40 Total Dissolved Solids  511 non-null float64
41 Month                   511 non-null object
42 Total Coliform          511 non-null float64
43 Flouride                494 non-null object
44 Conductivity            511 non-null float64
45 Human Activities        511 non-null object
46 Total Alkalinity        511 non-null float64
47 Temperature             511 non-null float64
48 Turbidity               511 non-null float64
49 Chlorides               511 non-null float64
50 Phosphate               511 non-null object
dtypes: float64(21), int64(2), object(28)
memory usage: 223.8+ KB

```

```
1 combined_df.columns[combined_df.dtypes == 'object'].tolist()
```

```

['Approx Depth',
 'Type Water Body',
 'latitude',
 'Name Of Water Body',
 'Use Based Class',
 'Sampling Date',
 'State Name',
 'Total Kjeldahl N',
 'Mon Agency',
 'Potassium',
 'Major Polluting Sources',
 'Sampling Time',
 'Weather',
 'Color',
 'Frequency',
 'District',
 'Visibility Effluent Discharge',
 'Floating Matter',
 'longitude',
 'River Basin',
 'Phenophelene Alkanity',
 'Nitrate N',
 'Stn Name',
 'Boron',
 'Month',
 'Flouride',
 'Human Activities',
 'Phosphate']

```

```

1 # l1 = ['Phosphate',
2 # 'Floating Matter',
3 # 'Total Kjeldahl N',
4 # 'Visibility Effluent Discharge',
5 # 'Major Polluting Sources',
6 # 'Nitrate N',
7 # 'Month',
8 # 'Sampling Time',
9 # 'latitude',
10 # 'Name Of Water Body',
11 # 'Type Water Body',
12 # 'Color',
13 # 'Phenophelene Alkanity',
14 # 'Mon Agency',
15 # 'Use Based Class',
16 # 'Potassium',
17 # 'Stn Name',
18 # 'Sampling Date',

```

```

19 # 'Approx Depth',
20 # 'Boron',
21 # 'District',
22 # 'longitude',
23 # 'River Basin',
24 # 'State Name',
25 # 'Frequency',
26 # 'Human Activities',
27 # 'Weather',
28 # 'Flouride']
29 # '''
30 # Purpose:
31 #     Iterate through given list of columns, display unique values, ask user whether
32 #     to examine the column, detect string-type values, ask whether to convert those
33 #     string values to null, apply changes if approved, and record all actions
34 #     performed for future function creation.
35
36 # Args:
37 #     df (DataFrame): Input DataFrame
38 #     columns_list (list): List of column names to process
39
40 # Return:
41 #     updated_df (DataFrame): DataFrame after applying string-to-null operations
42 #     '''
43 # def interactive_string_null_processing(df, columns_list):
44 #     updated_df = df.copy()
45 #     actions_log = {}
46
47 #     for col in columns_list:
48 #         if col not in updated_df.columns:
49 #             continue
50
51 #         print("\nColumn:", col)
52 #         uniques = updated_df[col].dropna().unique()
53 #         print("Unique Values:", uniques)
54
55 #         choice = input("List for examination y/n: ").strip().lower()
56
57 #         if choice == "y":
58 #             string_values = [val for val in uniques if isinstance(val, str)]
59
60 #             print("String Values Found:", string_values)
61 #             null_choice = input("Make all string values null? y/n: ").strip().lower()
62
63 #             if null_choice == "y":
64 #                 for val in string_values:
65 #                     updated_df[col] = updated_df[col].replace(val, None)
66 #                     actions_log[col] = {
67 #                         "action": "Converted strings to null",
68 #                         "values": string_values
69 #                     }
70 #             else:
71 #                 actions_log[col] = "Examined but skipped string-null conversion"
72 #         else:
73 #             actions_log[col] = "Skipped"
74
75 #         print("\nSummary of Actions:")
76 #         for col, action in actions_log.items():
77 #             print(f"{col} -> {action}")
78
79 #     return updated_df
80
81
82 # combined_df = interactive_string_null_processing(combined_df, l1)
83

```

```

1 '''
2 Purpose:
3     Apply predefined string-to-null replacements based on previously recorded
4     user actions, and additionally convert columns to float where no string
5     values were present but column dtype is still object.
6
7 Args:
8     df (DataFrame): Input DataFrame
9
10 Return:
11     updated_df (DataFrame): DataFrame after applying string-null replacements
12     and automatic float conversions
13 '''
14 def apply_string_null_and_float_fix(df):
15     import pandas as pd
16
17     updated_df = df.copy()
18
19     string_null_mapping = {
20         "Phosphate": ["ND"],
21         "Total Kjeldahl N": [],
22         "Nitrate N": ["ND"],
23         "Phenophelene Alkanity": [],
24         "Potassium": [],
25         "Boron": [],

```

```

26     "Flouride": ["ND"]
27 }
28
29 actions_log = {}
30
31 for col, string_values in string_null_mapping.items():
32     if col not in updated_df.columns:
33         continue
34
35     if len(string_values) > 0:
36         for val in string_values:
37             updated_df[col] = updated_df[col].replace(val, None)
38             actions_log[col] = {"action": "Converted strings to null", "values": string_values}
39     else:
40         numeric_version = pd.to_numeric(updated_df[col], errors="coerce")
41         updated_df[col] = numeric_version.combine_first(updated_df[col])
42         actions_log[col] = {"action": "Converted column to float"}
43
44 print("\nSummary of Actions:")
45 for col, action in actions_log.items():
46     print(f"{col} -> {action}")
47
48 return updated_df
49
50
51 combined_df = apply_string_null_and_float_fix(combined_df)
52

```

#### Summary of Actions:

```

Phosphate -> {'action': 'Converted strings to null', 'values': ['ND']}
Total Kjeldahl N -> {'action': 'Converted column to float'}
Nitrate N -> {'action': 'Converted strings to null', 'values': ['ND']}
Phenophelene Alkanity -> {'action': 'Converted column to float'}
Potassium -> {'action': 'Converted column to float'}
Boron -> {'action': 'Converted column to float'}
Flouride -> {'action': 'Converted strings to null', 'values': ['ND']}

```

```

1 '''
2 Purpose:
3     Force specified columns to numeric by cleaning strings,
4     coercing invalid values to NaN, and converting dtype to float.
5
6 Args:
7     df (DataFrame): Input DataFrame
8     columns (list): Columns to enforce numeric type
9
10 Return:
11     updated_df (DataFrame): DataFrame with cleaned numeric columns
12 '''
13 def force_columns_to_float(df, columns):
14     import pandas as pd
15
16     updated_df = df.copy()
17
18     for col in columns:
19         if col in updated_df.columns:
20             updated_df[col] = (
21                 updated_df[col]
22                 .astype(str)
23                 .str.strip()
24                 .replace(["nan", "None", ""], None)
25             )
26
27             updated_df[col] = pd.to_numeric(updated_df[col], errors="coerce")
28
29     return updated_df
30
31
32 numeric_columns = [
33     "Phosphate",
34     "Total Kjeldahl N",
35     "Nitrate N",
36     "Phenophelene Alkanity",
37     "Potassium",
38     "Boron",
39     "Flouride"
40 ]
41
42 combined_df = force_columns_to_float(combined_df, numeric_columns)
43

```

```
1 combined_df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
Index: 511 entries, 0 to 665
Data columns (total 51 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Approx Depth         511 non-null    object
1   Type Water Body      511 non-null    object
2   latitude             498 non-null    object
3   Name Of Water Body   498 non-null    object

```

```

3 name of water body          458 non-null object
4 Calcium CaCo3              511 non-null float64
5 Use Based Class             511 non-null object
6 Sampling Date               511 non-null object
7 STN Code                    511 non-null int64
8 State Name                  511 non-null object
9 Total Kjeldahl N            510 non-null float64
10 Mon Agency                 511 non-null object
11 Potassium                  510 non-null float64
12 Flow                       506 non-null float64
13 Major Polluting Sources    409 non-null object
14 Total Fixed Solids         511 non-null float64
15 BOD                        511 non-null float64
16 Sampling Time              511 non-null object
17 Weather                    511 non-null object
18 Color                      511 non-null object
19 Total Suspended Solids     511 non-null int64
20 Frequency                  511 non-null object
21 Amonia N                   507 non-null float64
22 District                   511 non-null object
23 Visibility Effluent Discharge 511 non-null object
24 Hardness CaCo3             511 non-null float64
25 Floating Matter            511 non-null object
26 longitude                  498 non-null object
27 River Basin                450 non-null object
28 Phenophelene Alkanity      507 non-null float64
29 Nitrate N                  507 non-null float64
30 Magnesium CaCo3           511 non-null float64
31 Sulphate                   511 non-null float64
32 Stn Name                   511 non-null object
33 Boron                      458 non-null float64
34 Fecal Coliform            511 non-null float64
35 COD                        511 non-null float64
36 Fecal Streptococci        511 non-null float64
37 Dissolved O2               511 non-null float64
38 pH                         511 non-null float64
39 Sodium                     511 non-null float64
40 Total Dissolved Solids     511 non-null float64
41 Month                      511 non-null object
42 Total Coliform             511 non-null float64
43 Flouride                   488 non-null float64
44 Conductivity               511 non-null float64
45 Human Activities           511 non-null object
46 Total Alkalinity           511 non-null float64
47 Temperature                511 non-null float64
48 Turbidity                  511 non-null float64
49 Chlorides                  511 non-null float64
50 Phosphate                  505 non-null float64
dtypes: float64(28), int64(2), object(21)
memory usage: 223.8+ KB

```

```

1 '''
2 Purpose:
3     Iterate through float64 columns, calculate missing percentage and skewness,
4     and fill missing values using mean if skewness is between -0.5 and 0.5,
5     otherwise fill using median.
6
7 Args:
8     df (DataFrame): Input DataFrame
9
10 Return:
11     updated_df (DataFrame): DataFrame with missing values imputed
12 '''
13 def impute_float_columns_based_on_skew(df):
14     import pandas as pd
15
16     updated_df = df.copy()
17     actions_log = {}
18
19     float_cols = updated_df.select_dtypes(include=["float64"]).columns
20
21     for col in float_cols:
22         missing_percent = (updated_df[col].isna().sum() / len(updated_df)) * 100
23         skewness = updated_df[col].skew()
24
25         if updated_df[col].isna().sum() > 0:
26             if -0.5 <= skewness <= 0.5:
27                 fill_value = updated_df[col].mean()
28                 updated_df[col] = updated_df[col].fillna(fill_value)
29                 actions_log[col] = {
30                     "missing_%": round(missing_percent, 2),
31                     "skewness": round(skewness, 2),
32                     "imputed_with": "mean"
33                 }
34             else:
35                 fill_value = updated_df[col].median()
36                 updated_df[col] = updated_df[col].fillna(fill_value)
37                 actions_log[col] = {
38                     "missing_%": round(missing_percent, 2),
39                     "skewness": round(skewness, 2),
40                     "imputed_with": "median"
41                 }
42
43     print("\nImputation Summary:")
44     for col, info in actions_log.items():
45         print(f"{col} -> {info}")

```

```

46
47     return updated_df
48
49
50 combined_df = impute_float_columns_based_on_skew(combined_df)
51

```

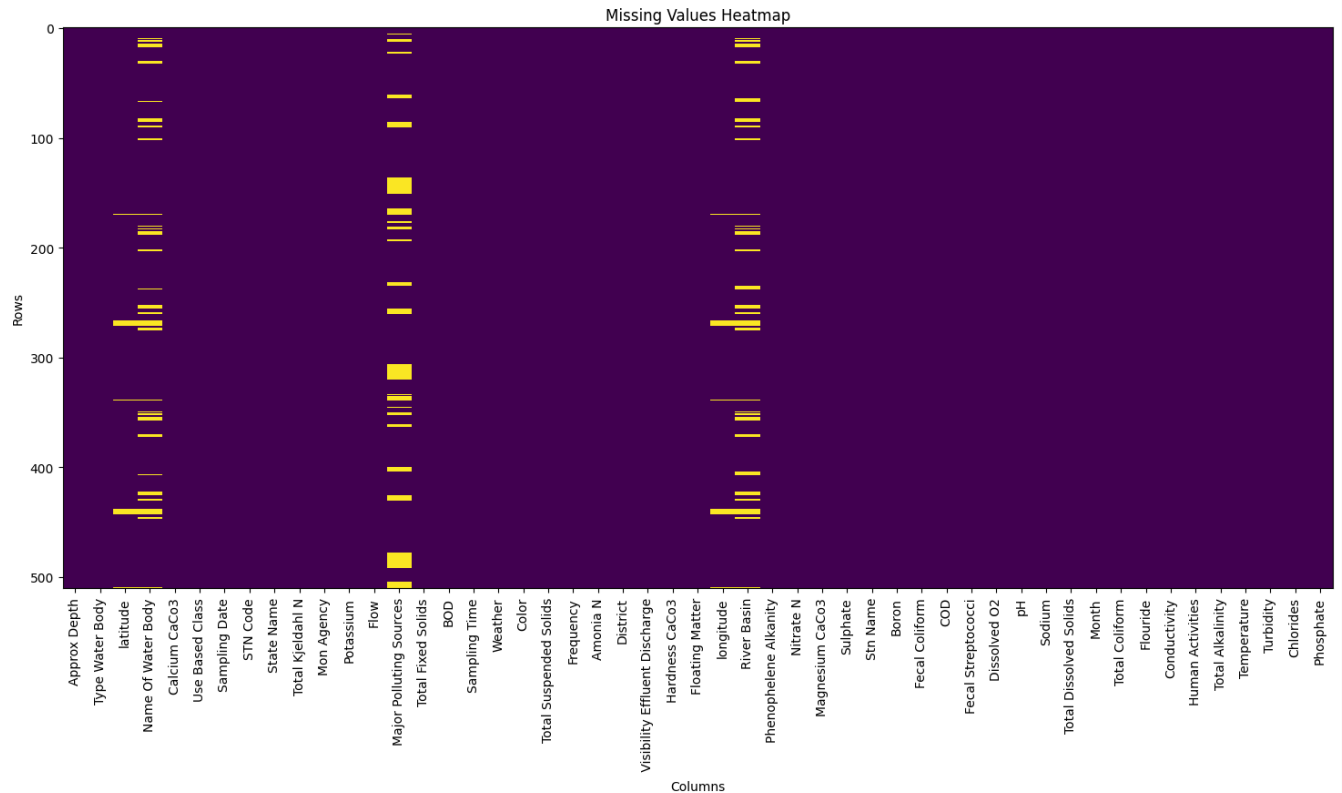
Imputation Summary:

```

Total Kjeldahl N -> {'missing_%': np.float64(0.2), 'skewness': np.float64(6.17), 'imputed_with': 'median'}
Potassium -> {'missing_%': np.float64(0.2), 'skewness': np.float64(4.42), 'imputed_with': 'median'}
Flow -> {'missing_%': np.float64(0.98), 'skewness': np.float64(3.35), 'imputed_with': 'median'}
Amonia N -> {'missing_%': np.float64(0.78), 'skewness': np.float64(6.66), 'imputed_with': 'median'}
Phenophelene Alkanity -> {'missing_%': np.float64(0.78), 'skewness': np.float64(12.28), 'imputed_with': 'median'}
Nitrate N -> {'missing_%': np.float64(0.78), 'skewness': np.float64(5.03), 'imputed_with': 'median'}
Boron -> {'missing_%': np.float64(10.37), 'skewness': np.float64(5.77), 'imputed_with': 'median'}
Flouride -> {'missing_%': np.float64(4.5), 'skewness': np.float64(4.64), 'imputed_with': 'median'}
Phosphate -> {'missing_%': np.float64(1.17), 'skewness': np.float64(4.27), 'imputed_with': 'median'}

```

```
1 plot_missing_values_heatmap(combined_df)
```



## ✖ Duplication handling

```

1 '''
2 Purpose:
3     Check for duplicate rows in the DataFrame and print the count of duplicates.
4
5 Args:
6     df (DataFrame): Input DataFrame
7
8 Return:
9     None
10 '''
11 def check_duplicates(df):
12     duplicate_count = df.duplicated().sum()
13     print(f"Total duplicate rows: {duplicate_count}")
14
15
16 check_duplicates(combined_df)
17

```

Total duplicate rows: 0

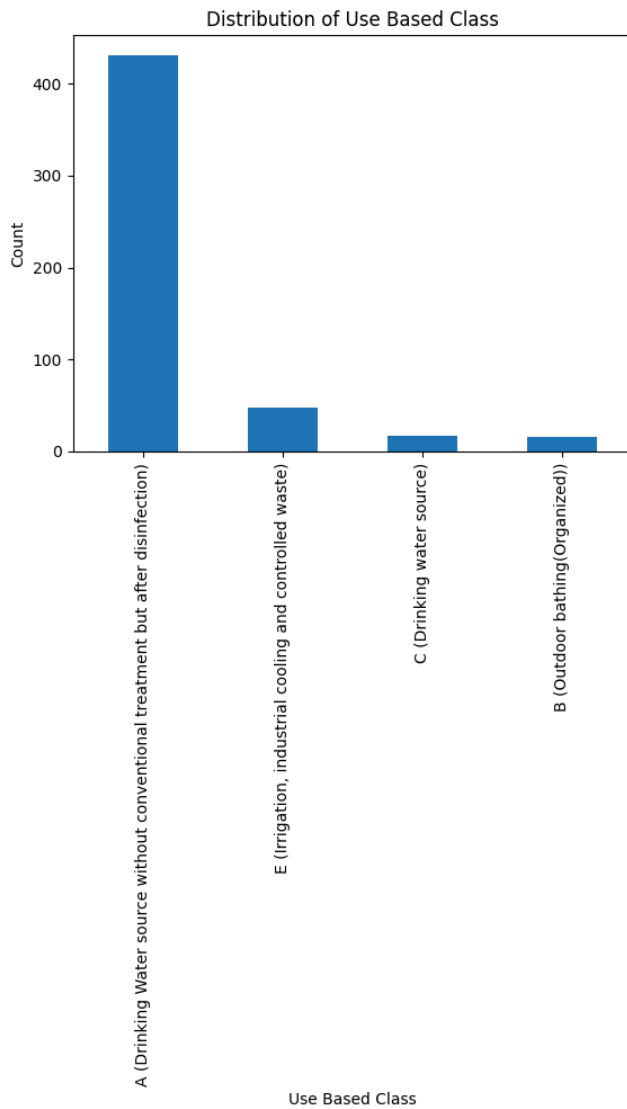
## ✖ 2. Exploratory Data Analysis Plots

### ✖ Chart 1

```

1 import matplotlib.pyplot as plt
2
3 counts = combined_df["Use Based Class"].value_counts()
4
5 plt.figure()
6 counts.plot(kind="bar")
7 plt.xlabel("Use Based Class")
8 plt.ylabel("Count")
9 plt.title("Distribution of Use Based Class")
10 plt.show()
11

```



**Plot:** Count Plot – Use Based Class

**Why to choose this plot:** To understand the distribution of target classes and check whether the dataset is imbalanced before training a machine learning model.

**What info I got:** Most samples belong to Class A (Drinking water source without conventional treatment but after disinfection), while Classes B, C, and E have much fewer records, indicating strong class imbalance.

## chart 2

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 cols = ["pH", "BOD", "Dissolved O2", "COD"]
5
6 for col in cols:
7     plt.figure()
8     sns.histplot(combined_df[col], kde=True)
9     plt.title(f"Distribution of {col}")
10    plt.xlabel(col)
11    plt.ylabel("Frequency")

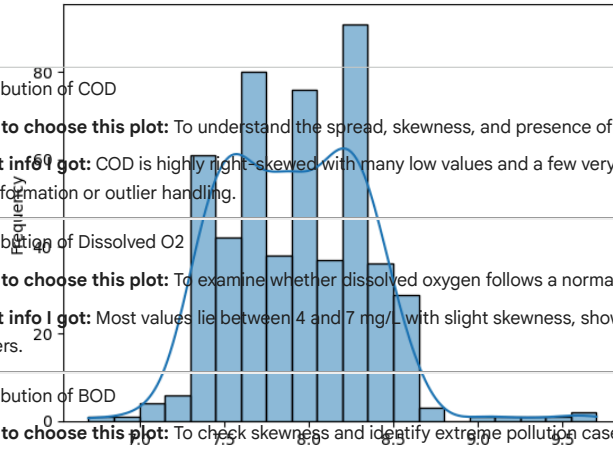
```

```
12 plt.show()  
13
```





Distribution of pH



Distribution of COD

**Why to choose this plot:** To understand the spread, skewness, and presence of extreme values in COD before modeling.

**What info I got:** COD is highly right-skewed with many low values and a few very high outliers, indicating possible need for log transformation or outlier handling.

Distribution of Dissolved O2

**Why to choose this plot:** To examine whether dissolved oxygen follows a normal pattern and detect unusual values.

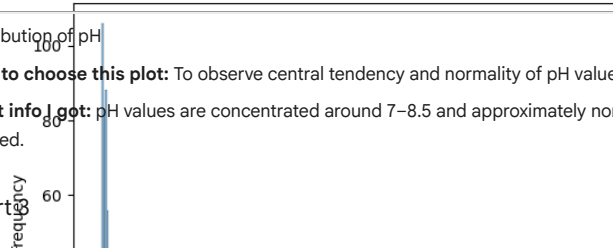
**What info I got:** Most values lie between 4 and 7 mg/L with slight skewness, showing relatively stable oxygen levels and fewer extreme outliers.

Distribution of BOD

**Why to choose this plot:** To check skewness and identify extreme pollution cases affecting model learning.

**What info I got:** BOD is strongly right-skewed with many small values and some very large values, suggesting outlier treatment or transformation is required.

Distribution of BOD



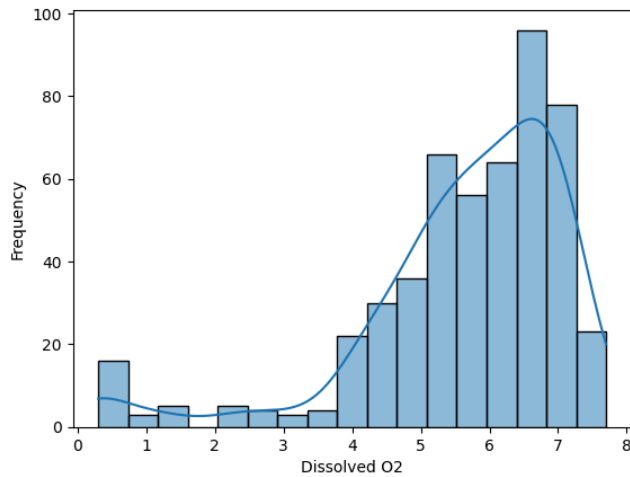
Distribution of pH

**Why to choose this plot:** To observe central tendency and normality of pH values.

**What info I got:** pH values are concentrated around 7–8.5 and approximately normally distributed, indicating minimal transformation is needed.

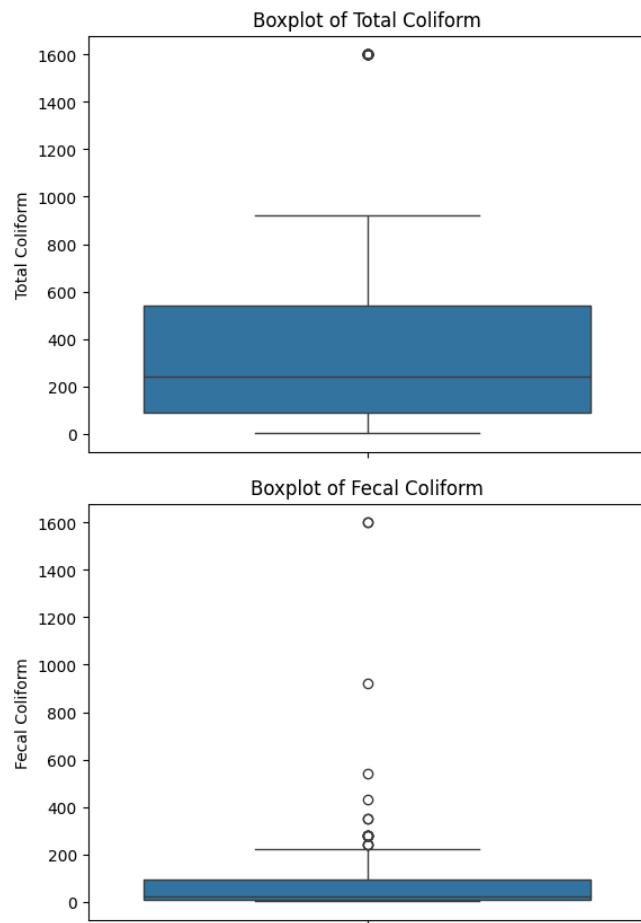
charts

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 cols = ["Total Coliform", "Fecal Coliform"]
5
6 for col in cols:
7     plt.figure()
8     sns.boxplot(y=combined_df[col])
9     plt.title(f"Boxplot of {col}")
10    plt.show()
11
```



Distribution of COD





Boxplot of Total Coliform

**Why to choose this plot:** To detect outliers and understand the spread of total coliform levels, which indicate overall bacterial contamination.

**What info I got:** Most values lie at lower levels, but there are extreme high outliers, showing occasional heavy contamination events that may require outlier handling.

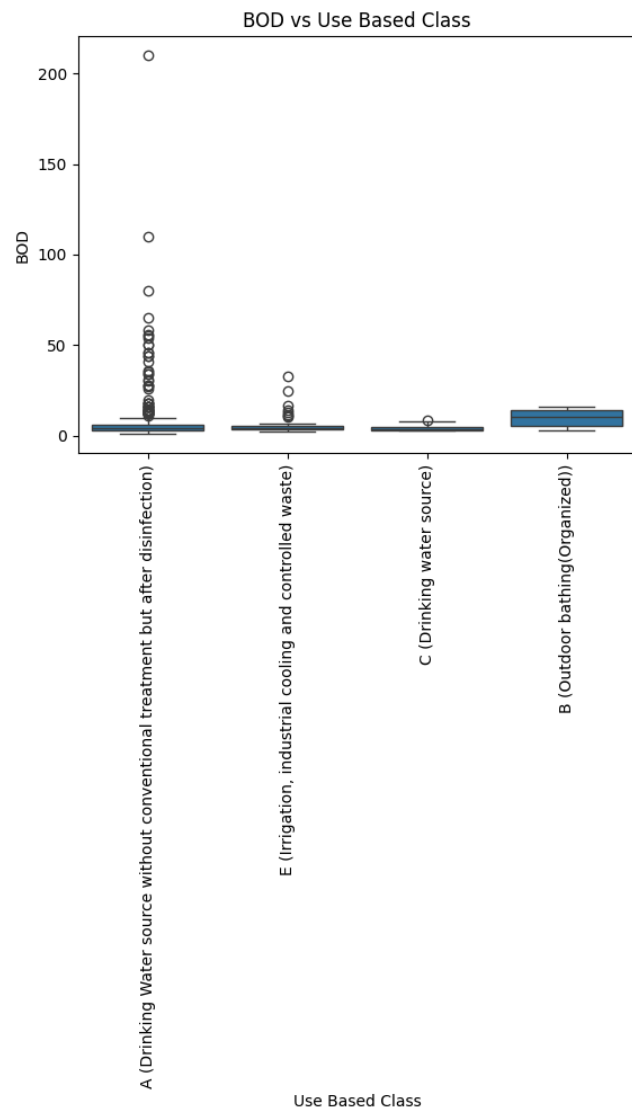
Boxplot of Fecal Coliform

**Why to choose this plot:** To identify extreme fecal contamination values and check data spread.

**What info I got:** Fecal coliform values are highly right-skewed with several large outliers, suggesting strong variability and need for transformation or capping.

#### chart 4

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 plt.figure()
5 sns.boxplot(x="Use Based Class", y="BOD", data=combined_df)
6 plt.title("BOD vs Use Based Class")
7 plt.xticks(rotation=90)
8 plt.show()
9
```



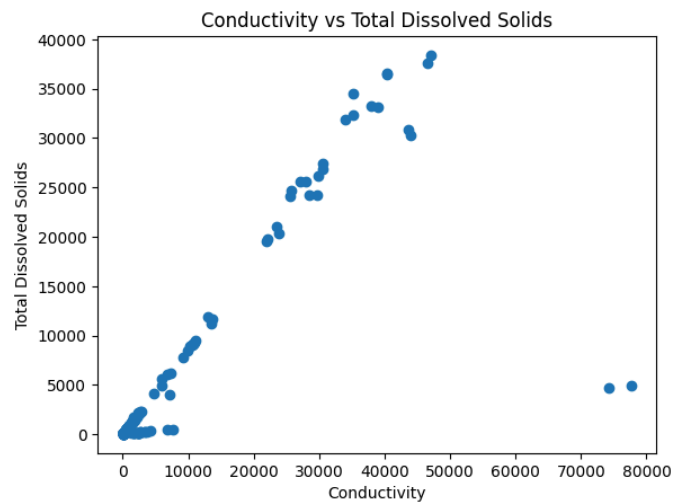
Box Plot of BOD vs Use Based Class

Why to choose this plot: To compare how Biochemical Oxygen Demand (BOD) varies across different water use classes and evaluate its ability to differentiate between classes.

What info I got: BOD distributions differ across classes, with higher median and wider spread for lower-quality classes, indicating that BOD is an important feature for predicting water use classification.

## chart 5

```
1 import matplotlib.pyplot as plt
2
3 plt.figure()
4 plt.scatter(combined_df["Conductivity"], combined_df["Total Dissolved Solids"])
5 plt.xlabel("Conductivity")
6 plt.ylabel("Total Dissolved Solids")
7 plt.title("Conductivity vs Total Dissolved Solids")
8 plt.show()
9
```



Scatter Plot of Conductivity vs Total Dissolved Solids

Why to choose this plot: To check the relationship and correlation between conductivity and total dissolved solids and identify redundancy between features.

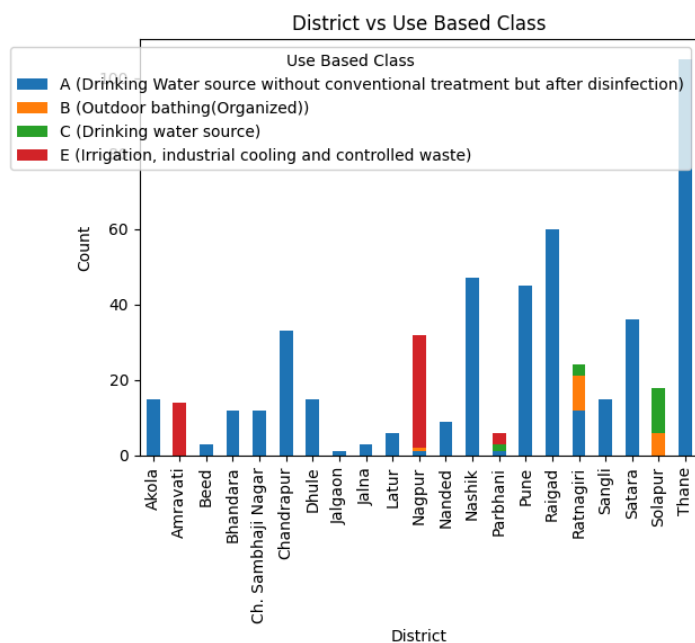
What info I got: There is a strong positive linear relationship between conductivity and total dissolved solids, suggesting that one of these features could be dropped to reduce multicollinearity.

## chart 6

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 cross_tab = pd.crosstab(combined_df["District"], combined_df["Use Based Class"])
5
6 cross_tab.plot(kind="bar", stacked=True)
7 plt.xlabel("District")
8 plt.ylabel("Count")
9 plt.title("District vs Use Based Class")
10 plt.show()
11

```



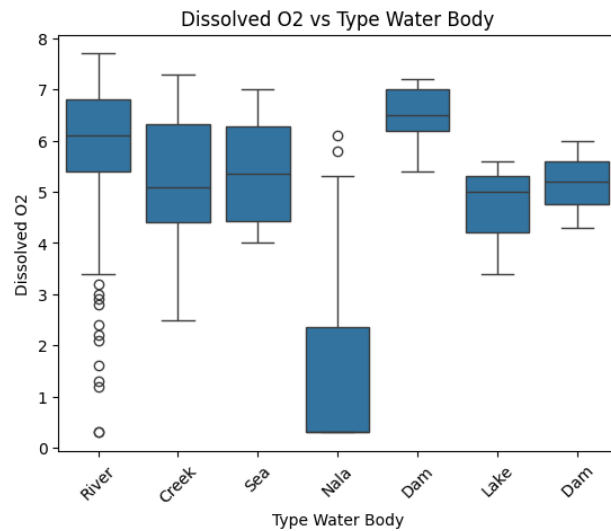
Bar Chart of District vs Use Based Class

Why to choose this plot: To analyze how water use classes vary across different districts and detect geographic patterns in water quality.

What info I got: Some districts have a higher concentration of lower-quality classes compared to others, indicating location plays an important role in determining water quality and can be a useful predictive feature.

## chart 7

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 plt.figure()
5 sns.boxplot(x="Type Water Body", y="Dissolved O2", data=combined_df)
6 plt.xticks(rotation=45)
7 plt.title("Dissolved O2 vs Type Water Body")
8 plt.show()
9
```



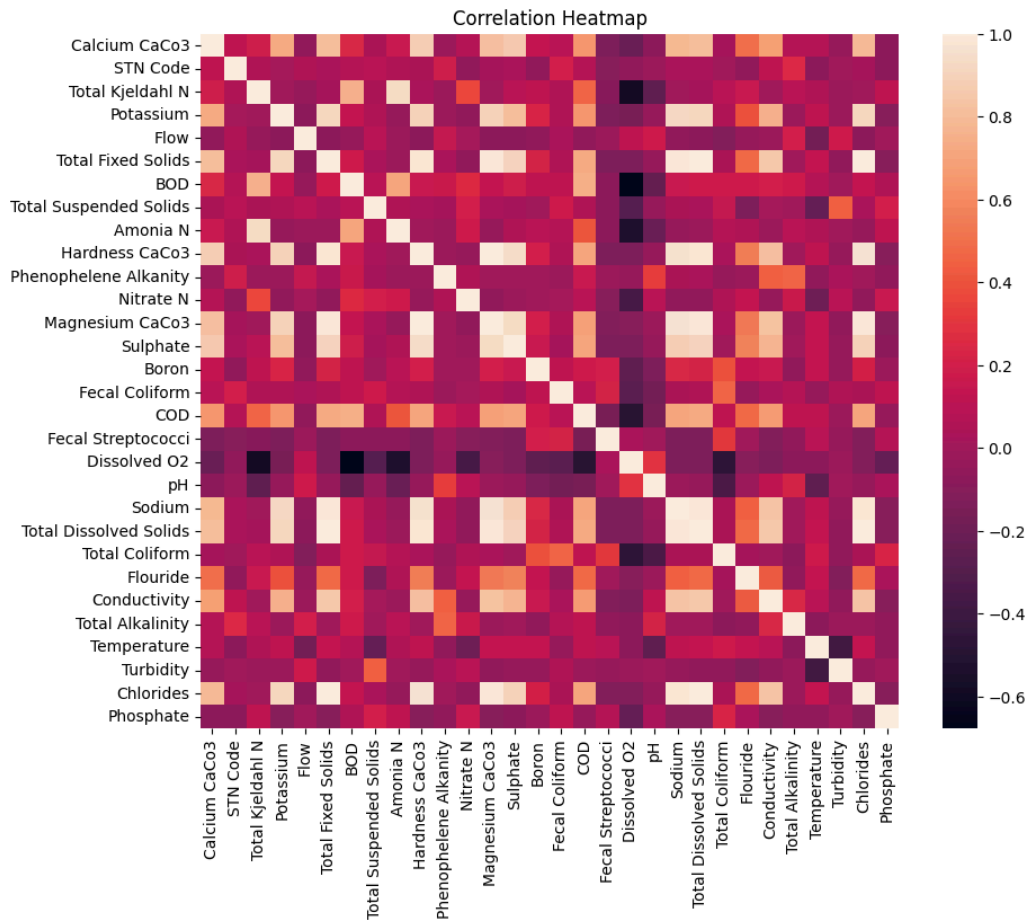
Box Plot of Dissolved O2 vs Type Water Body

Why to choose this plot: To compare dissolved oxygen levels across different types of water bodies and understand how water body type influences oxygen availability.

What info I got: Dams and rivers show higher median dissolved oxygen levels compared to lakes and creeks, indicating that water body type significantly affects oxygen concentration and is a useful categorical feature for modeling.

## chart 8

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 numeric_df = combined_df.select_dtypes(include=["float64", "int64"])
5 corr = numeric_df.corr()
6
7 plt.figure(figsize=(10,8))
8 sns.heatmap(corr)
9 plt.title("Correlation Heatmap")
10 plt.show()
11
```



#### Correlation Heatmap (Numeric Features)

Why to choose this plot: To understand relationships between numeric features and detect highly correlated variables that may cause multicollinearity.

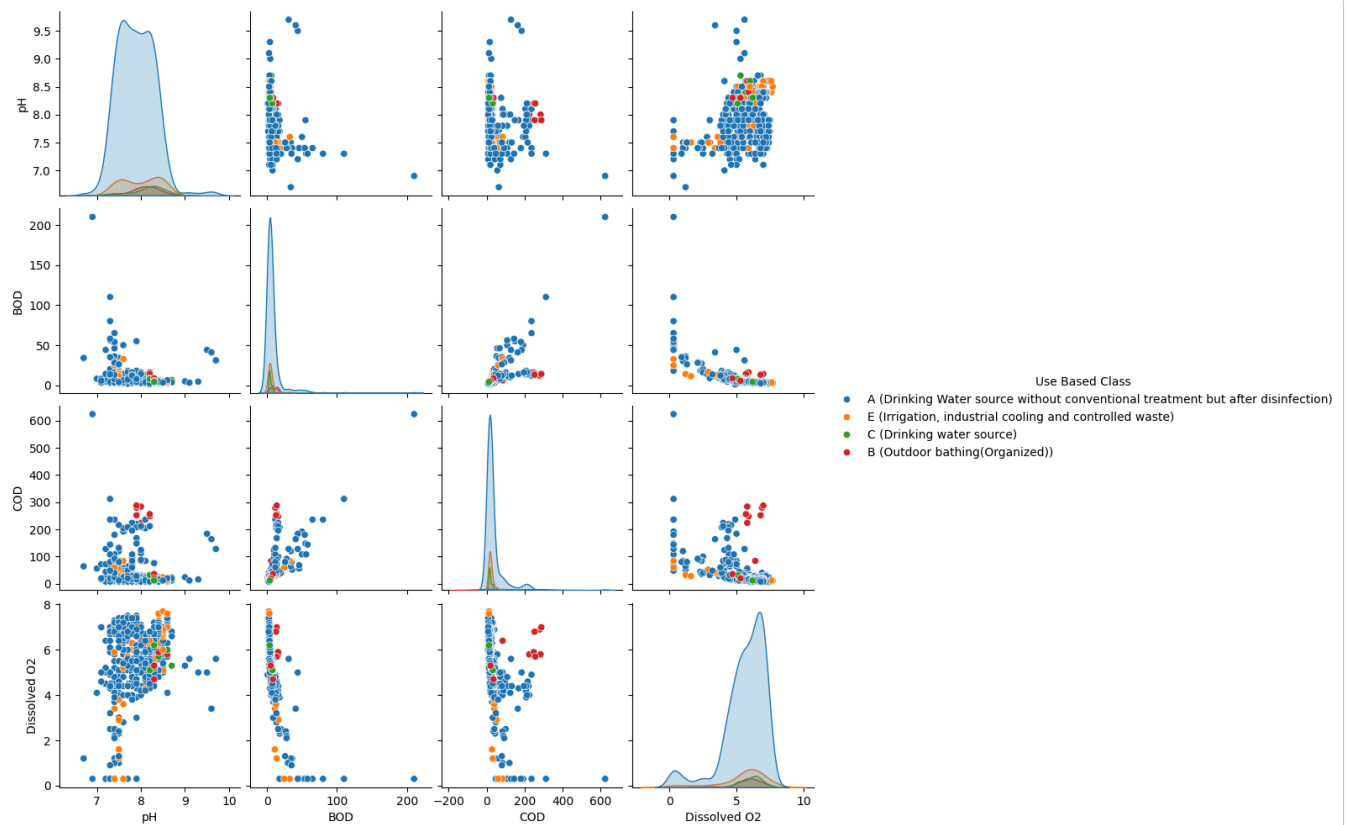
What info I got: Several physicochemical parameters show moderate to strong positive correlations (e.g., conductivity with total dissolved solids, sodium with chlorides), indicating redundancy among some features and guiding feature selection.

#### chart 9

```

1 import seaborn as sns
2
3 features = ["pH", "BOD", "COD", "Dissolved O2", "Use Based Class"]
4 sns.pairplot(combined_df[features], hue="Use Based Class")
5 plt.show()
6

```



Pair Plot (pH, BOD, COD, Dissolved O2 vs Use Based Class)

**Why to choose this plot:** To visualize relationships between key water quality parameters simultaneously and observe how different use-based classes separate across multiple feature combinations.

**What info I got:** Some class separation is visible, especially for BOD and COD, while pH and Dissolved O2 show overlapping distributions. This suggests non-linear boundaries and indicates that tree-based models (e.g., Random Forest) may perform better than simple linear models.

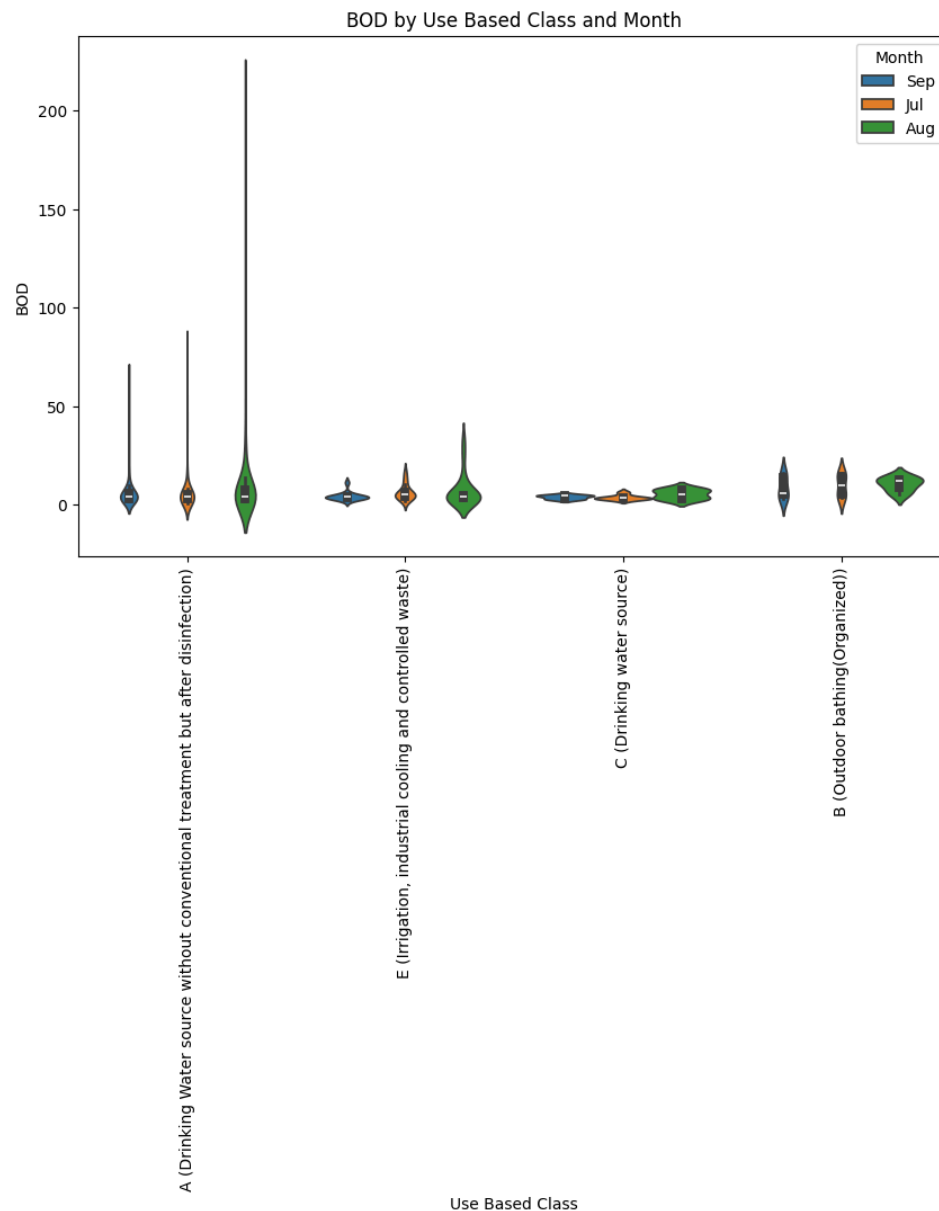
## chart 10

```

1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 plt.figure(figsize=(10,6))
5 sns.violinplot(x="Use Based Class", y="BOD", hue="Month", data=combined_df)
6 plt.title("BOD by Use Based Class and Month")
7 plt.xticks(rotation=90)
8 plt.show()
9

```





Violin Plot of BOD by Use Based Class Split by Month

**Why to choose this plot:** To analyze seasonal variation of BOD within each water use class and understand whether month influences pollution levels.

**What info I got:** BOD distributions vary slightly across months for the same class, indicating seasonal effects exist. This suggests that `Month` is a meaningful feature and should be included in the machine learning model.

### 3. Feature Engineering

#### Feature Selection Based On EDA

```
1 combined_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 511 entries, 0 to 665
Data columns (total 51 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Approx Depth         511 non-null    object
1   Type Water Body      511 non-null    object
2   latitude             498 non-null    object
3   Name Of Water Body   456 non-null    object
4   Calcium CaCo3        511 non-null    float64
5   Use Based Class      511 non-null    object
6   Sampling Date        511 non-null    object
7   STN Code             511 non-null    int64
8   State Name           511 non-null    object
9   Total Chlorophyll a  511 non-null    float64
```

```

9 Total Kjeldahl N          511 non-null float64
10 Mon Agency               511 non-null object
11 Potassium                511 non-null float64
12 Flow                    511 non-null float64
13 Major Polluting Sources 409 non-null object
14 Total Fixed Solids       511 non-null float64
15 BOD                     511 non-null float64
16 Sampling Time           511 non-null object
17 Weather                 511 non-null object
18 Color                   511 non-null object
19 Total Suspended Solids  511 non-null int64
20 Frequency               511 non-null object
21 Amonia N                511 non-null float64
22 District                511 non-null object
23 Visibility Effluent Discharge 511 non-null object
24 Hardness CaCo3          511 non-null float64
25 Floating Matter         511 non-null object
26 longitude               498 non-null object
27 River Basin             450 non-null object
28 Phenophelene Alkanity   511 non-null float64
29 Nitrate N               511 non-null float64
30 Magnesium CaCo3        511 non-null float64
31 Sulphate                511 non-null float64
32 Stn Name                511 non-null object
33 Boron                   511 non-null float64
34 Fecal Coliform          511 non-null float64
35 COD                     511 non-null float64
36 Fecal Streptococci     511 non-null float64
37 Dissolved O2            511 non-null float64
38 pH                      511 non-null float64
39 Sodium                  511 non-null float64
40 Total Dissolved Solids  511 non-null float64
41 Month                   511 non-null object
42 Total Coliform          511 non-null float64
43 Flouride                511 non-null float64
44 Conductivity            511 non-null float64
45 Human Activities        511 non-null object
46 Total Alkalinity        511 non-null float64
47 Temperature             511 non-null float64
48 Turbidity               511 non-null float64
49 Chlorides               511 non-null float64
50 Phosphate               511 non-null float64
dtypes: float64(28), int64(2), object(21)
memory usage: 223.8+ KB

```

```

1 '''
2 Purpose:
3     Create a new DataFrame by selecting the final chosen feature columns
4     for machine learning and dropping redundant features.
5
6 Args:
7     df (DataFrame): Input root DataFrame
8
9 Return:
10    selected_df (DataFrame): DataFrame containing only selected features
11 '''
12 def create_selected_feature_df(df):
13     selected_columns = [
14         "pH",
15         "Dissolved O2",
16         "BOD",
17         "COD",
18         "Turbidity",
19         "Temperature",
20         "Conductivity",
21         "Chlorides",
22         "Total Dissolved Solids",
23         "Hardness CaCo3",
24         "Nitrate N",
25         # "Total Kjeldahl N",
26         "Phosphate",
27         "Amonia N",
28         "Total Coliform",
29         "Fecal Coliform",
30         # "Type Water Body",
31         "Use Based Class"
32     ]
33
34     selected_df = df[selected_columns].copy()
35
36     return selected_df
37
38
39 final_df = create_selected_feature_df(combined_df)
40

```

## Feature Selection Rationale for Water Quality Classification

The objective of this project is to classify water sources into multiple *Use Based Classes* using physicochemical and microbiological parameters. Although the dataset contains a large number of variables, using all features is neither efficient nor beneficial for machine learning model performance. Therefore, a reduced and carefully selected feature set is constructed based on domain knowledge and exploratory data analysis (EDA).

### 1. Importance of Feature Reduction

High-dimensional datasets often introduce noise, redundancy, and multicollinearity, which can:

- Reduce model interpretability
- Increase training time
- Lead to overfitting
- Decrease generalization performance

Selecting fewer but highly informative features improves model stability and predictive power.

---

## 2. Selection Based on Domain Knowledge

Water quality classification primarily depends on indicators representing:

- Acidity/alkalinity (pH)
- Organic pollution (BOD, COD)
- Oxygen availability (Dissolved O<sub>2</sub>)
- Nutrient pollution (Nitrate N, Total Kjeldahl N, Phosphate, Ammonia N)
- Salinity and mineral content (Conductivity, Chlorides, Total Dissolved Solids)
- Microbial contamination (Total Coliform, Fecal Coliform)
- Physical condition (Turbidity, Temperature)
- Environmental context (Type Water Body)

These parameters are widely accepted in environmental science as strong indicators of water quality.

---

## 3. Selection Based on Correlation Analysis

Correlation heatmap analysis revealed strong relationships between certain variables:

- Conductivity and Total Dissolved Solids
- Sodium and Chlorides
- Total Suspended Solids and Total Fixed Solids

To avoid redundancy and multicollinearity, only one representative feature from each highly correlated pair is retained.

Kept:

- Conductivity
- Chlorides
- Total Dissolved Solids

Dropped:

- Sodium
  - Total Suspended Solids
  - Total Fixed Solids
- 

## 4. Selection Based on Exploratory Data Analysis

From histograms, boxplots, scatter plots, and pair plots:

- BOD, COD, Dissolved O<sub>2</sub>, and pH show meaningful variation across Use Based Classes.
- Bacterial indicators contain strong outliers but remain critical predictors.
- Type Water Body influences dissolved oxygen distribution and indirectly affects water quality.

These findings support retaining these variables in the final feature set.

---

## 5. Final Selected Features

The final model input consists of the following columns:

- pH
- Dissolved O<sub>2</sub>
- BOD
- COD
- Turbidity
- Temperature
- Conductivity
- Chlorides
- Total Dissolved Solids
- Hardness CaCO<sub>3</sub>
- Nitrate N
- Total Kjeldahl N
- Phosphate
- Ammonia N
- Total Coliform
- Fecal Coliform
- Type Water Body

Target Variable:

- Use Based Class
-

## 6. Expected Benefits

This reduced feature set:

- Minimizes noise and redundancy
- Improves model training speed
- Enhances generalization capability
- Preserves interpretability
- Aligns with environmental science standards

Thus, the selected features provide an optimal balance between model simplicity and predictive strength for water quality classification.

## Encoding

```
1 # # 16 Type Water Body 511 non-null object
2 # # 17 Use Based Class 511 non-null object
3
4 # final_df['Type Water Body'].unique()
```

```
1 final_df['Use Based Class'].unique()
```

```
array(['A (Drinking Water source without conventional treatment but after disinfection)',
       'E (Irrigation, industrial cooling and controlled waste)',
       'C (Drinking water source)', 'B (Outdoor bathing(Organized))'],
      dtype=object)
```

```
1 '''
2 Purpose:
3     Apply value-based encoding for target column and clean + encode
4     Type Water Body column.
5
6 Args:
7     df (DataFrame): Input DataFrame
8
9 Return:
10    updated_df (DataFrame): Encoded DataFrame
11 '''
12 def apply_value_encodings(df):
13     updated_df = df.copy()
14
15     updated_df["Use Based Class"] = updated_df["Use Based Class"].map({
16         "A (Drinking Water source without conventional treatment but after disinfection)": 0,
17         "B (Outdoor bathing(Organized))": 1,
18         "C (Drinking water source)": 2,
19         "E (Irrigation, industrial cooling and controlled waste)": 3
20     })
21
22     # updated_df["Type Water Body"] = updated_df["Type Water Body"].replace({
23     #     " Dam": "Dam"
24     # })
25
26     # updated_df["Type Water Body"] = updated_df["Type Water Body"].map({
27     #     "River": 0,
28     #     "Creek": 1,
29     #     "Sea": 2,
30     #     "Nala": 3,
31     #     "Dam": 4,
32     #     "Lake": 5
33     # })
34
35     return updated_df
36
37
38 final_df = apply_value_encodings(final_df)
39
```

## Feature Selection

```
1 final_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 511 entries, 0 to 665
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   pH                    511 non-null    float64
1   Dissolved O2          511 non-null    float64
2   BOD                   511 non-null    float64
3   COD                   511 non-null    float64
4   Turbidity             511 non-null    float64
5   Temperature           511 non-null    float64
6   Conductivity          511 non-null    float64
7   Chlorides             511 non-null    float64
8   Total Dissolved Solids 511 non-null    float64
9   Hardness CaCo3        511 non-null    float64
10  Nitrate N             511 non-null    float64
11  Phosphate             511 non-null    float64
```

```

12 Amonia N          511 non-null    float64
13 Total Coliform    511 non-null    float64
14 Fecal Coliform    511 non-null    float64
15 Use Based Class   511 non-null    int64
dtypes: float64(15), int64(1)
memory usage: 84.0 KB

```

```

1 '''
2 Purpose:
3     Calculate Variance Inflation Factor (VIF) for each numeric feature
4     and return results ordered from highest to lowest VIF.
5
6 Args:
7     df (DataFrame): Input DataFrame containing numeric features
8
9 Return:
10    vif_df (DataFrame): VIF table sorted in descending order
11 '''
12 def calculate_vif(df):
13     import pandas as pd
14     from statsmodels.stats.outliers_influence import variance_inflation_factor
15
16     X = df.select_dtypes(include=["float64", "int64"])
17
18     vif_data = []
19     for i in range(X.shape[1]):
20         vif_data.append(
21             {
22                 "Feature": X.columns[i],
23                 "VIF": variance_inflation_factor(X.values, i)
24             }
25         )
26
27     vif_df = pd.DataFrame(vif_data)
28     vif_df = vif_df.sort_values(by="VIF", ascending=False).reset_index(drop=True)
29
30     return vif_df
31
32
33 vif_df = calculate_vif(final_df)
34 vif_df
35

```

|    | Feature                | VIF        |  |
|----|------------------------|------------|--|
| 0  | Total Dissolved Solids | 162.746433 |  |
| 1  | Chlorides              | 121.977885 |  |
| 2  | pH                     | 109.747830 |  |
| 3  | Temperature            | 62.915043  |  |
| 4  | Dissolved O2           | 39.852242  |  |
| 5  | Hardness CaCo3         | 30.153700  |  |
| 6  | COD                    | 18.892000  |  |
| 7  | BOD                    | 12.968167  |  |
| 8  | Conductivity           | 4.555585   |  |
| 9  | Total Coliform         | 3.144744   |  |
| 10 | Amonia N               | 2.929591   |  |
| 11 | Nitrate N              | 2.549431   |  |
| 12 | Phosphate              | 2.225957   |  |
| 13 | Fecal Coliform         | 1.784127   |  |
| 14 | Use Based Class        | 1.441092   |  |
| 15 | Turbidity              | 1.256121   |  |

Next steps: [Generate code with vif\\_df](#) [New interactive sheet](#)

## Split train / test

```

1 '''
2 Purpose:
3     Split dataset into training and testing sets with stratified distribution.
4
5 Args:
6     df (DataFrame): Input DataFrame
7     target_column (str): Name of target column
8     test_size (float): Proportion of test data
9
10 Return:
11     X_train, X_test, y_train, y_test
12 '''

```

```

13 def stratified_train_test_split(df, target_column, test_size=0.3):
14     from sklearn.model_selection import train_test_split
15
16     X = df.drop(columns=[target_column])
17     y = df[target_column]
18
19     X_train, X_test, y_train, y_test = train_test_split(
20         X,
21         y,
22         test_size=test_size,
23         stratify=y,
24         random_state=42
25     )
26
27     return X_train, X_test, y_train, y_test
28
29
30 X_train, X_test, y_train, y_test = stratified_train_test_split(
31     final_df,
32     "Use Based Class",
33     0.3
34 )
35

```

## 4. Machine Learning Model Training

| Model                               | Train F1 Score | Test F1 Score | Train Confusion Matrix Behavior                              | Test Confusion Matrix Behavior                                    |
|-------------------------------------|----------------|---------------|--------------------------------------------------------------|-------------------------------------------------------------------|
| <b>SVM (Scaled)</b>                 | 0.6809         | 0.6799        | Moderate diagonal dominance, heavy confusion between A and C | Similar confusion between A and C, weak minority class separation |
| <b>Logistic Regression (Scaled)</b> | 0.5640         | 0.5690        | Weak diagonal, many misclassifications across classes        | Poor separation of classes, struggles with non-linearity          |
| <b>Random Forest (Tuned)</b>        | 0.9918         | 0.8425        | Almost perfect diagonal → strong memorization                | Good diagonal dominance, some confusion in minority classes       |
| <b>LightGBM (Tuned)</b>             | 1.0000         | 0.8750        | Perfect diagonal → complete training fit                     | Strong diagonal dominance, best minority class recognition        |

## Random Forest

```

1 '''
2 Purpose:
3     Train a baseline Random Forest classifier and display confusion matrix
4     plots with class names and F1-score for both training and testing datasets.
5
6 Args:
7     X_train (DataFrame): Training features
8     X_test (DataFrame): Testing features
9     y_train (Series): Training labels
10    y_test (Series): Testing labels
11
12 Return:
13    model (RandomForestClassifier): Trained model
14 '''
15 def train_random_forest_and_evaluate(X_train, X_test, y_train, y_test):
16     import matplotlib.pyplot as plt
17     from sklearn.ensemble import RandomForestClassifier
18     from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, f1_score
19
20     class_labels = [
21         "A Drinking",
22         "B outdoor bathing",
23         "C Drinking H BOD",
24         "E Irrigation industry"
25     ]
26
27     model = RandomForestClassifier(random_state=42)
28     model.fit(X_train, y_train)
29
30     y_train_pred = model.predict(X_train)
31     y_test_pred = model.predict(X_test)
32
33     print("Train F1 Score:", f1_score(y_train, y_train_pred, average="weighted"))
34     print("Test F1 Score:", f1_score(y_test, y_test_pred, average="weighted"))
35
36     train_cm = confusion_matrix(y_train, y_train_pred)
37     test_cm = confusion_matrix(y_test, y_test_pred)
38
39     plt.figure()
40     ConfusionMatrixDisplay(train_cm, display_labels=class_labels).plot()
41     plt.title("Train Confusion Matrix")
42     plt.xticks(rotation=90)
43     plt.show()
44
45     plt.figure()
46     ConfusionMatrixDisplay(test_cm, display_labels=class_labels).plot()
47     plt.title("Test Confusion Matrix")
48     plt.xticks(rotation=90)
49     plt.show()
50
51     return model
52

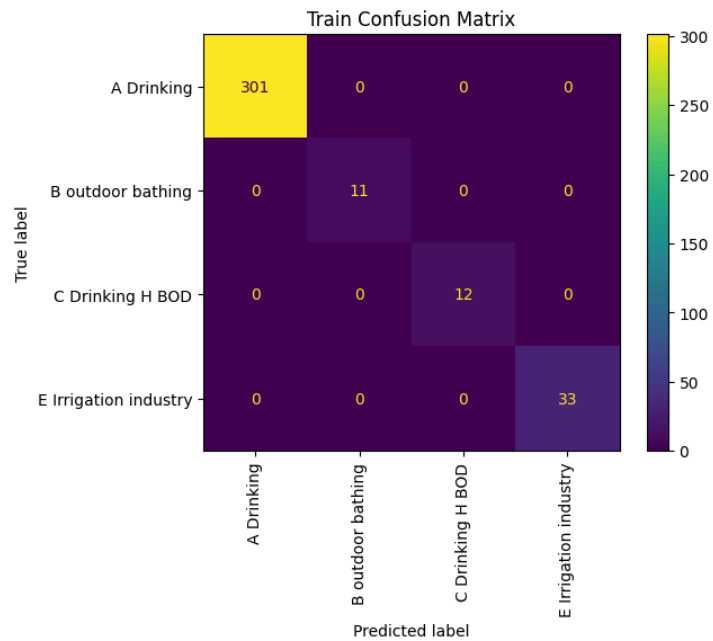
```

```

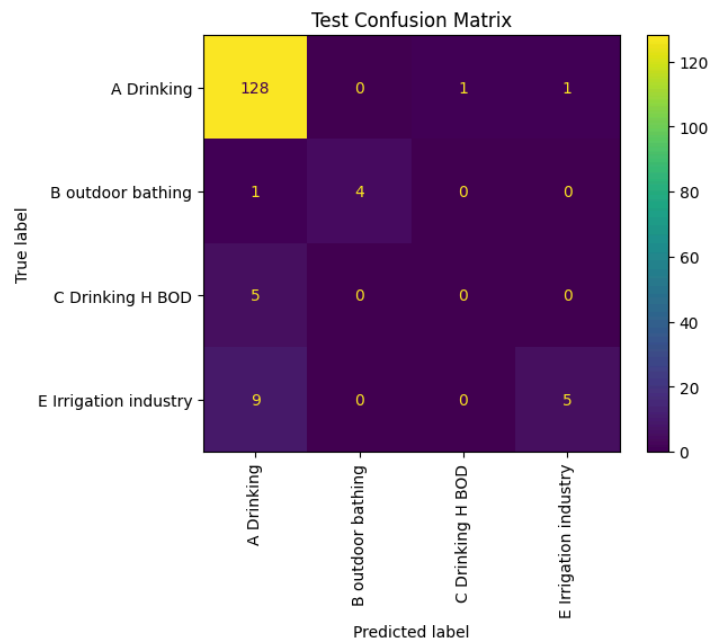
53
54 rf_model = train_random_forest_and_evaluate(
55     X_train,
56     X_test,
57     y_train,
58     y_test
59 )
60

```

Train F1 Score: 1.0  
 Test F1 Score: 0.8659039373325086  
 <Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>



## RandomCV hyperparameter optimization

```

1 '''
2 Purpose:
3     Perform RandomizedSearchCV to tune Random Forest hyperparameters
4     with class imbalance handling and F1-score optimization.
5
6 Args:
7     X_train (DataFrame): Training features
8     y_train (Series): Training labels
9
10 Return:
11     best_model (RandomForestClassifier): Best tuned model
12 '''
13 def randomcv_random_forest_f1_imbalance(X_train, y_train):

```

```

14 from sklearn.ensemble import RandomForestClassifier
15 from sklearn.model_selection import RandomizedSearchCV
16
17 param_grid = {
18     "n_estimators": [200, 300, 400, 500],
19     "max_depth": [8, 10, 12, 15, 20],
20     "min_samples_split": [5, 10, 15],
21     "min_samples_leaf": [2, 5, 10],
22     "max_features": ["sqrt", "log2"]
23 }
24
25 rf = RandomForestClassifier(
26     random_state=42,
27     class_weight="balanced"
28 )
29
30 random_search = RandomizedSearchCV(
31     estimator=rf,
32     param_distributions=param_grid,
33     n_iter=30,
34     scoring="f1_weighted",
35     cv=5,
36     verbose=1,
37     random_state=42,
38     n_jobs=-1
39 )
40
41 random_search.fit(X_train, y_train)
42
43 print("Best Parameters:")
44 print(random_search.best_params_)
45
46 return random_search.best_estimator_
47
48
49 best_rf_model = randomcv_random_forest_f1_imbalance(X_train, y_train)
50

```

Fitting 5 folds for each of 30 candidates, totalling 150 fits

Best Parameters:

```
{'n_estimators': 400, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2', 'max_depth': 8}
```

```

1 '''
2 Purpose:
3     Train Random Forest model using user-provided hyperparameters
4     and evaluate with confusion matrix plots and F1-score.
5
6 Args:
7     X_train (DataFrame): Training features
8     X_test (DataFrame): Testing features
9     y_train (Series): Training labels
10    y_test (Series): Testing labels
11    params (dict): Dictionary of Random Forest hyperparameters
12
13 Return:
14     model (RandomForestClassifier): Trained model
15 '''
16 def train_rf_with_given_params(X_train, X_test, y_train, y_test, params):
17     import matplotlib.pyplot as plt
18     from sklearn.ensemble import RandomForestClassifier
19     from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, f1_score
20
21     class_labels = [
22         "A Drinking",
23         "B outdoor bathing",
24         "C Drinking H BOD",
25         "E Irrigation industry"
26     ]
27
28     model = RandomForestClassifier(
29         random_state=42,
30         class_weight="balanced",
31         **params
32     )
33
34     model.fit(X_train, y_train)
35
36     y_train_pred = model.predict(X_train)
37     y_test_pred = model.predict(X_test)
38
39     print("Train F1 Score:", f1_score(y_train, y_train_pred, average="weighted"))
40     print("Test F1 Score:", f1_score(y_test, y_test_pred, average="weighted"))
41
42     train_cm = confusion_matrix(y_train, y_train_pred)
43     test_cm = confusion_matrix(y_test, y_test_pred)
44
45     plt.figure()
46     ConfusionMatrixDisplay(train_cm, display_labels=class_labels).plot()
47     plt.title("Train Confusion Matrix")
48     plt.show()
49

```



```

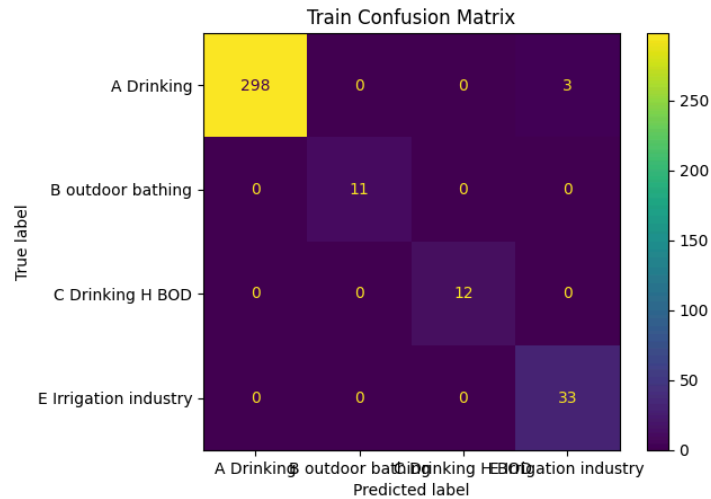
50 plt.figure()
51 ConfusionMatrixDisplay(test_cm, display_labels=class_labels).plot()
52 plt.title("Test Confusion Matrix")
53 plt.show()
54
55 return model
56
57
58 params = {'n_estimators': 400, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'log2', 'max_depth': 8}
59
60 final_rf_model = train_rf_with_given_params(
61     X_train,
62     X_test,
63     y_train,
64     y_test,
65     params
66 )
67

```

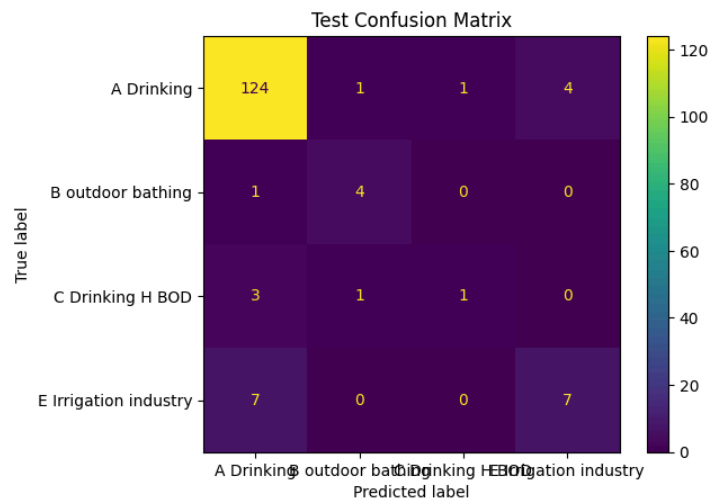
Train F1 Score: 0.9917582769480008

Test F1 Score: 0.8738007300222445

<Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>



## LightGBM

```

1 '''
2 Purpose:
3     Train a baseline LightGBM classifier and display confusion matrix
4     plots and F1-score for both training and testing datasets.
5
6 Args:
7     X_train (DataFrame): Training features
8     X_test (DataFrame): Testing features
9     y_train (Series): Training labels
10    y_test (Series): Testing labels
11
12 Return:
13     model (LGBMClassifier): Trained LightGBM model
14 '''
15 def train_lightgbm_baseline(X_train, X_test, y_train, y_test):

```

```
16 import matplotlib.pyplot as plt
17 from lightgbm import LGBMClassifier
18 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, f1_score
19
20 class_labels = ["A", "B", "C", "E"]
21
22 model = LGBMClassifier(random_state=42)
23 model.fit(X_train, y_train)
24
25 y_train_pred = model.predict(X_train)
26 y_test_pred = model.predict(X_test)
27
28 print("Train F1 Score:", f1_score(y_train, y_train_pred, average="weighted"))
29 print("Test F1 Score:", f1_score(y_test, y_test_pred, average="weighted"))
30
31 train_cm = confusion_matrix(y_train, y_train_pred)
32 test_cm = confusion_matrix(y_test, y_test_pred)
33
34 plt.figure()
35 ConfusionMatrixDisplay(train_cm, display_labels=class_labels).plot()
36 plt.title("Train Confusion Matrix")
37 plt.show()
38
39 plt.figure()
40 ConfusionMatrixDisplay(test_cm, display_labels=class_labels).plot()
41 plt.title("Test Confusion Matrix")
42 plt.show()
43
44 return model
45
46
47 lgbm_model = train_lightgbm_baseline(
48     X_train,
49     X_test,
50     y_train,
51     y_test
52 )
53
```

